

计算机辅助设计与制造

Computer Aided Design & Manufacturing



目 录

CONTENT

01

工程数据处理技术

02

图形处理技术



CAD&CAM

2⁰₂6

工程数据处理技术

2、Data Process

常用的表示方法有数表、线图

数表

Numerical Table
离散的列表数据

数
据

线图

Line Graph
直线/折线/曲线

数表 (Numerical Table)

1 理论或经验计算公式

通常用一个或一组计算公式表示，常以表格形式出现，便于检索和使用。

2 简单数表(独立的常量)

根据表中数据与自变量个数不同，分为一维数表、二维数表、多维数表。

3 列表函数数表

通常是通过试验方式测得的一组离散数据，这些互相对应的数据之间可能存在某种函数关系，但无法以明确的函数表达式描述。



01

线图 (Line Graph)

1

各参数之间存在较复杂的计算公式

将公式转换成线图以供设计时查用。

2

各参数之间没有或者不存在计算公式

如齿形系数

简单数表的程序化处理

以数组形式记录数表数据，程序运行时输入自变量，通过循环查得该自变量对应的数组下标

线图的表格化处理

在线图的横坐标上取一系列离散点，得到对应线图上的函数值。由此可以将线图离散成一个数表。

数字化

02

处理方法

列表函数数表的插值处理

不仅需要查询与自变量对应的因变量数据，还需要查询各自变量在节点区间内的对应值。需要采用**插值**（Interpolation）方法。

线图的公式化处理

若是直线线图，直接将其转化为线性方程，可直接求得函数值；若是曲线线图，可采用曲线拟合的方法求出线图曲线的经验公式。

数据文件的生成与检索

应用文本编辑软件直接编辑生成、利用高级语言中的文件读写语句编程序来生成。
对于无序文件采用顺序遍历算法、对于有序文件采用分段搜索算法。

数据结构（Data Structure）

计算机中数据元素的各种存储结构就是数据元素在计算机中的不同映像方法。
常用的数据结构：线性列表、栈结构、树形结构、二叉树结构。

数据文件

03

数据库

工程数据的文件化处理

先将数据以一定的次序存储在数据文件中，之后再编制数表处理程序。

数据库技术

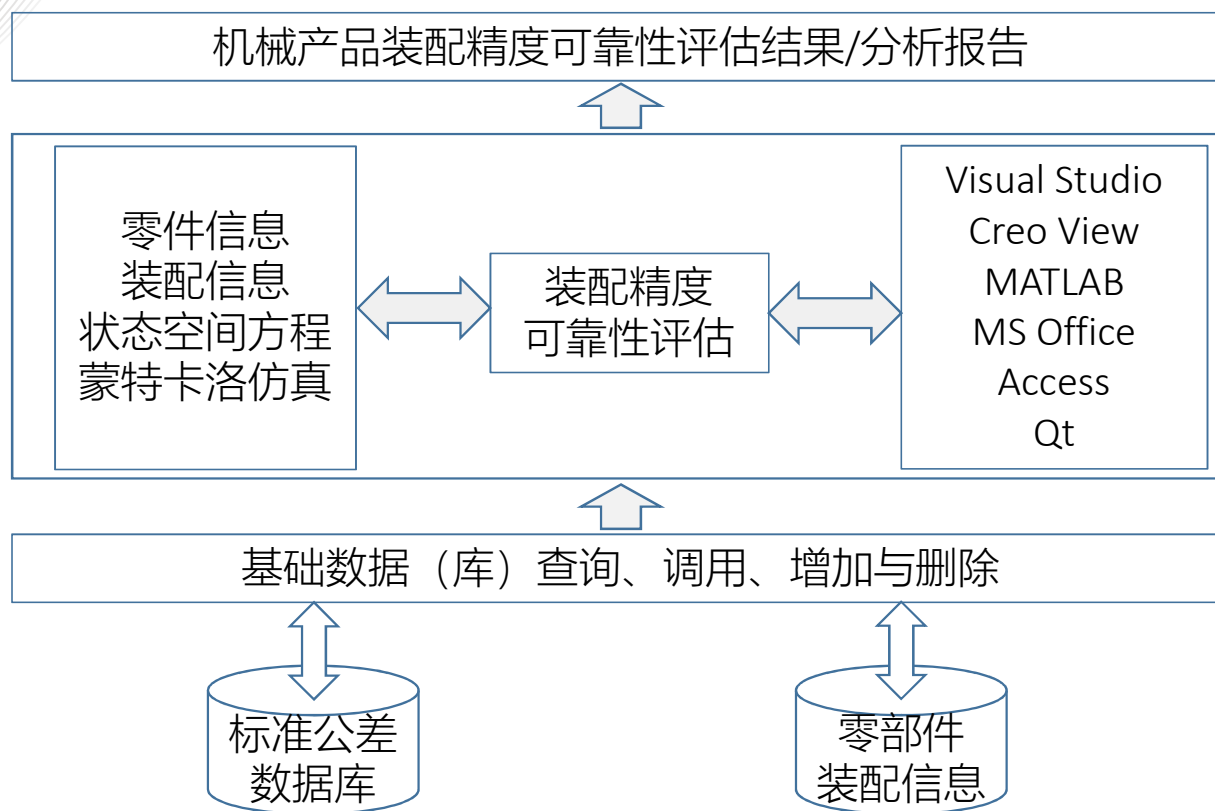
客观事物的多种属性都可以用数据加以描述，而数据库是数据管理的有效手段。
常用的数据模型：层次模型、网络模型、关系模型。

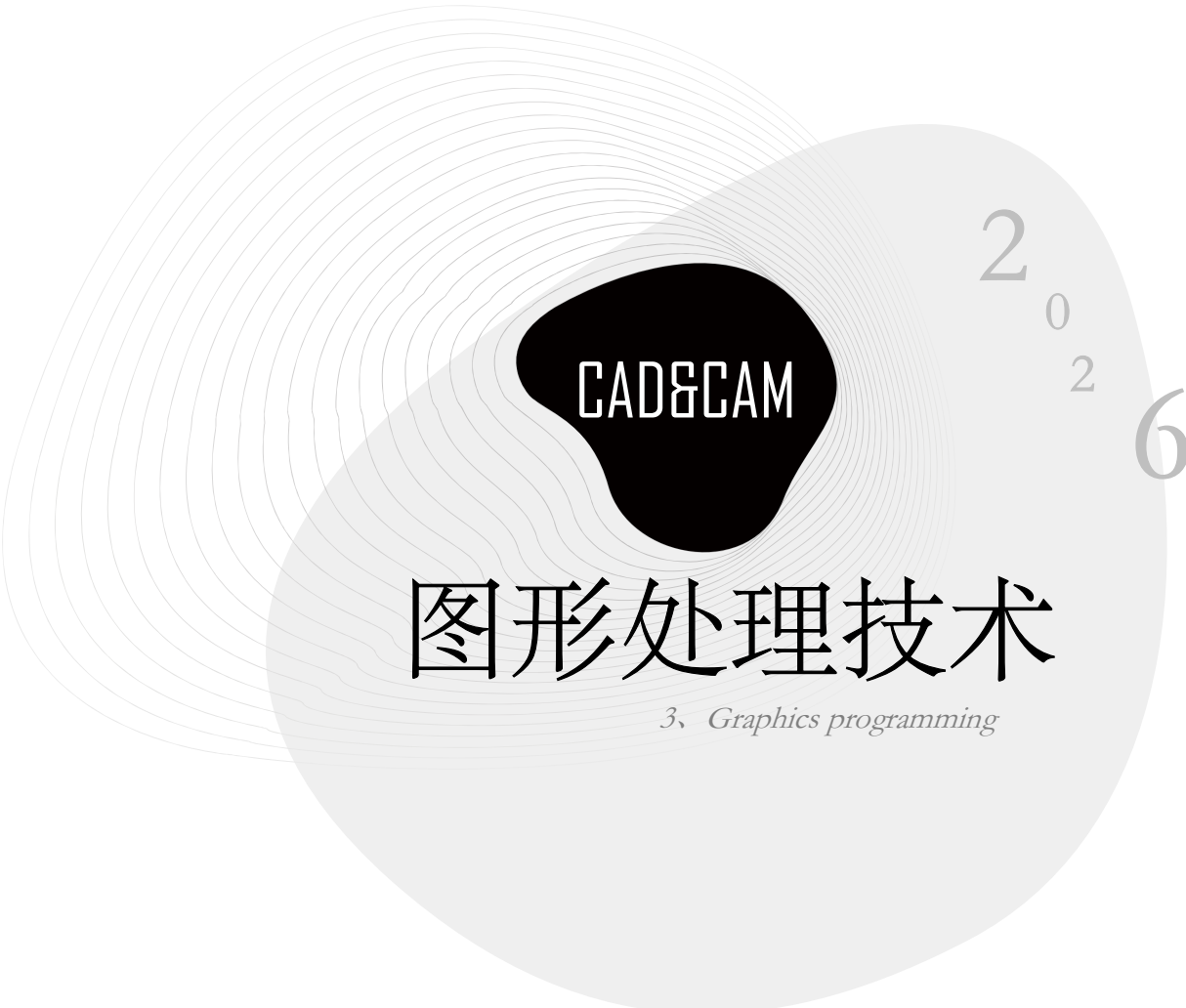
04 软件开发案例

CASE STUDY

结合某型产品的开发需求，采用状态空间方程完成装配误差建模，利用蒙特卡洛仿真方法评估产品装配精度可靠度；在保证装配精度评估准确性的基础上，对装配精度计算过程和评估方法做出必要简化，在此基础上开发机械产品装配精度评估软件。

• 机械产品 装配精度 评估软件





CAD&CAM

图形处理技术

3、*Graphics programming*

2⁰₂6

目录

CONTENT

- 01 计算机图形学概述
Graphics libraries
- 02 坐标系统
Coordinate systems
- 03 图形变换
Transformation matrix
- 04 图形裁剪
hidden-surface removal
- 05 图形绘制
Rendering

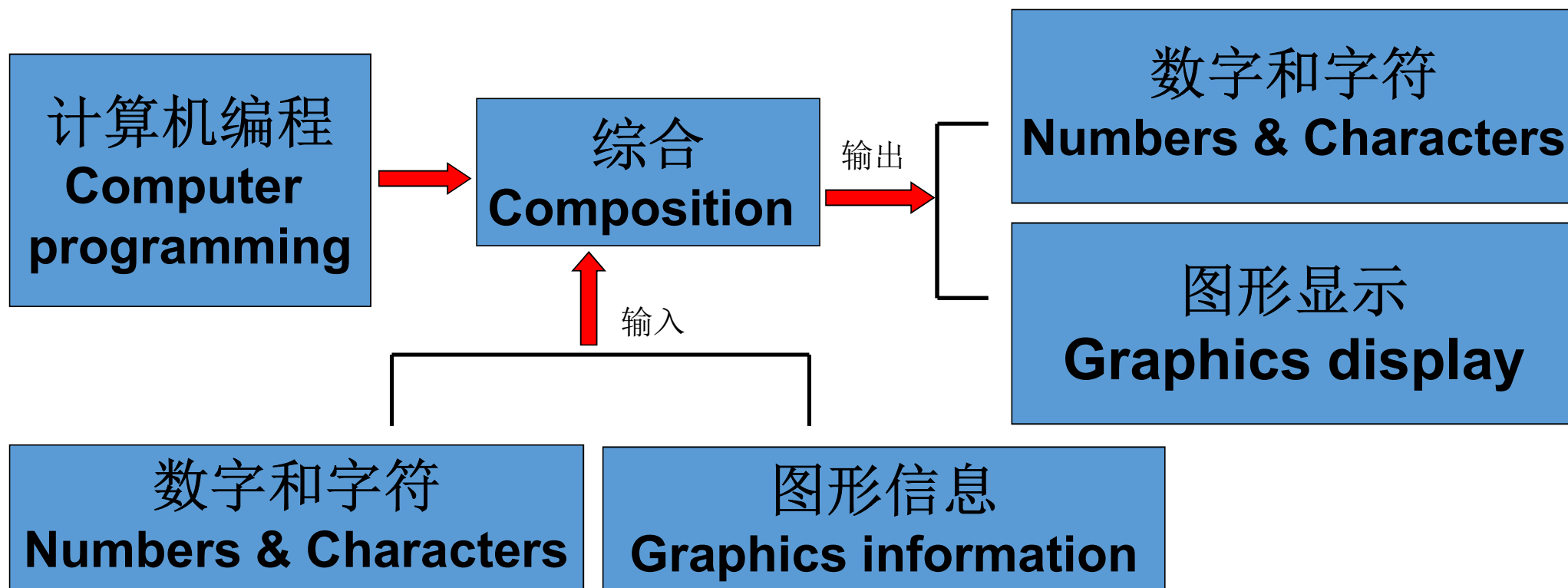
计算机图形学概述

■ 计算机图形学（Computer Graphics, CG）的研究内容

- **矢量图形**是由计算机记录图形的形状参数、颜色、线型等属性而形成的图形。
- **点阵图形**则是用点阵的充填来表示图形，构成图形的点都具有一定的灰度和色彩。

- ## ■ 手工绘图与计算机绘图：
- （1）图形输入功能
 - （2）存储功能
 - （3）计算功能
 - （4）显示和输出功能
 - （5）对话功能

图形设计与计算机图形学



目录

CONTENT

- 01 计算机图形学概述
Graphics libraries
- 02 坐标系统
Coordinate systems
- 03 图形变换
Transformation matrix
- 04 图形裁剪
hidden-surface removal
- 05 图形绘制
Rendering

02 坐标系统 Coordinate systems

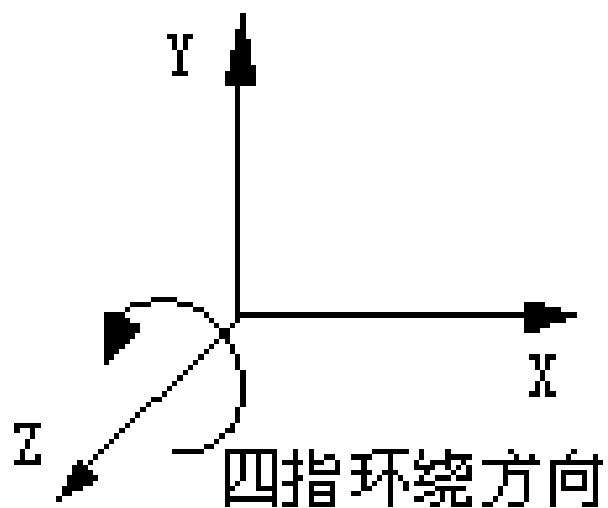
在图形设备上显示对象的图像需要完成两个基本任务：

- 1) 确定空间中所有点的位置；
- 2) 确定显示器上的哪些位置将被这些点占据。

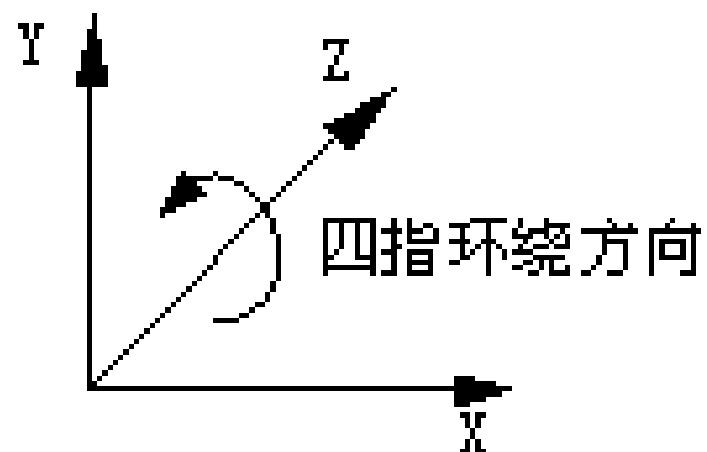
坐标系统提供了指定空间和显示器上位置的参考

02 坐标系

三维直角坐标系的定义



(a) 右手坐标系

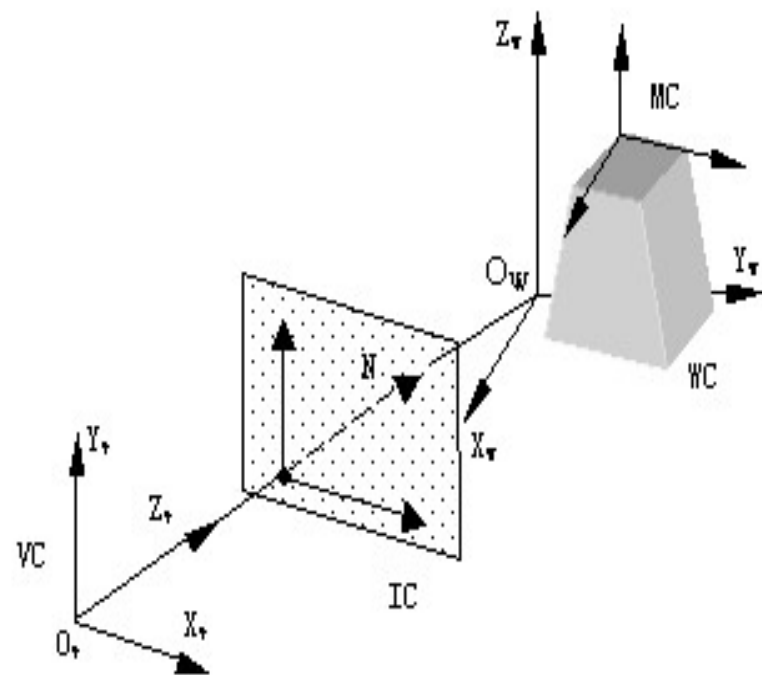


(b) 左手坐标系

02 坐标系统

- (1) 世界坐标系 (World Coordinate System, WC)
- (2) 建模坐标系 (Modeling Coordinate System, MC)
- (3) 观察坐标系 (Viewing Coordinate Systems, VC)
- (4) 成像坐标系 (Imaging Coordinate Systems, IC)
- (5) 设备坐标系 (Device Coordinate System, DC)
- (6) 虚拟设备坐标系 (Virtual Device Coordinate Systems, VDC)

图形系统中定义的坐标系



世界坐标系、建模坐标系和观察坐标系 World, model and view coordinate system

- **世界坐标系：**

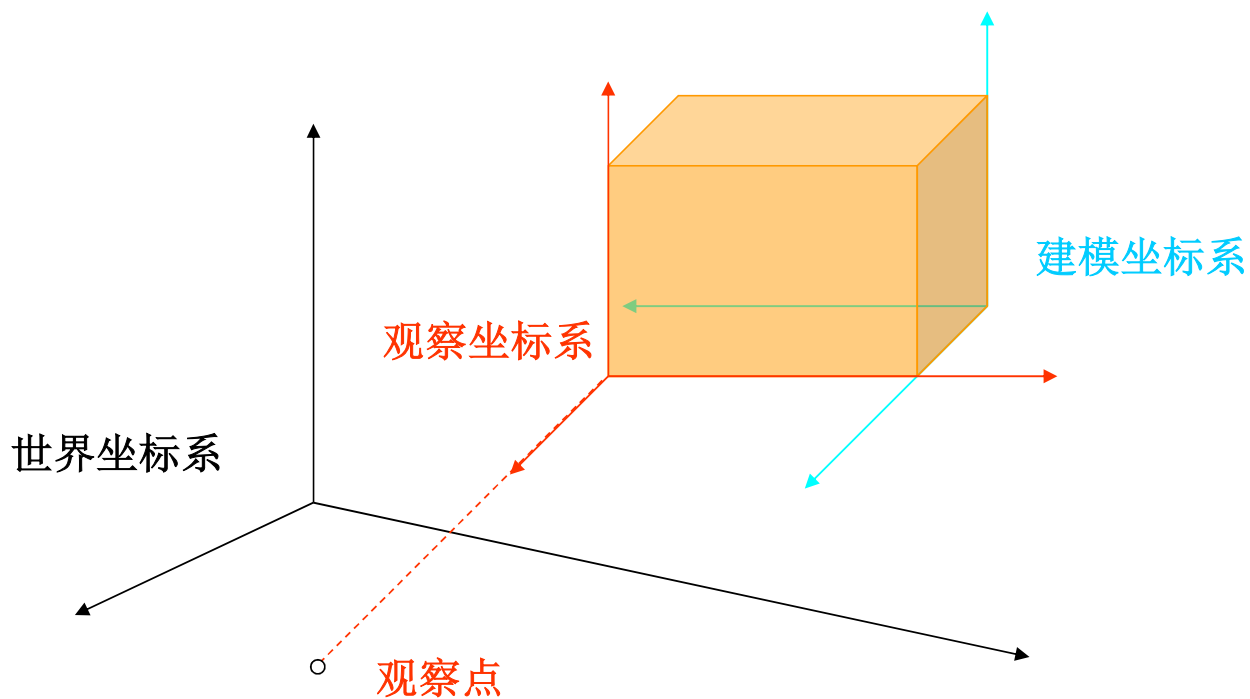
用于描述现实世界的整体布局，即各种对象在我们所描述的世界之中的定位。

- **建模坐标系：**

用于描述世界坐标系中每个具体物体的形状，每个物体均由其自身的建模坐标系定义。

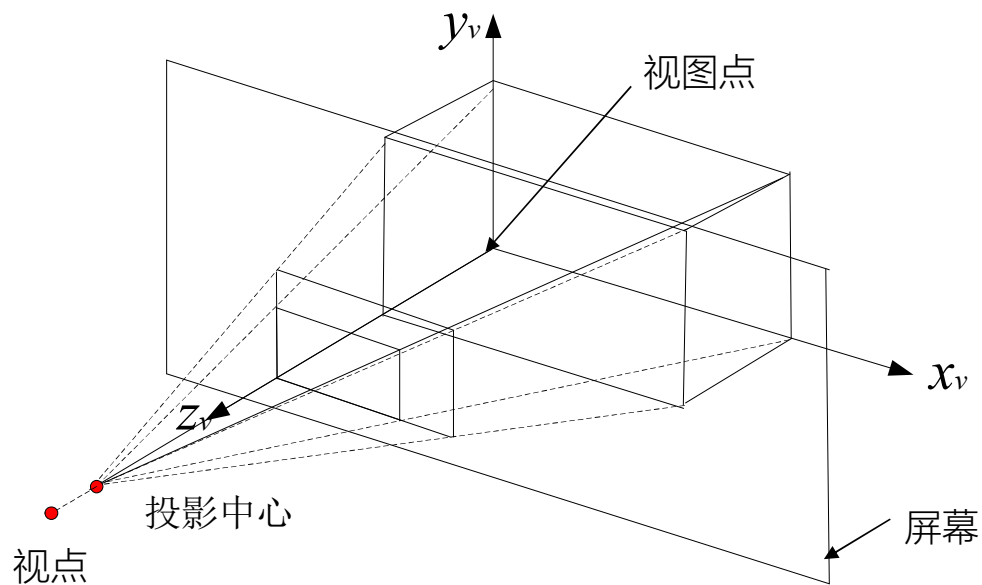
- **观察坐标系：**

观察坐标系由投影方式决定，通常采用透视投影和平行投影两种方式。

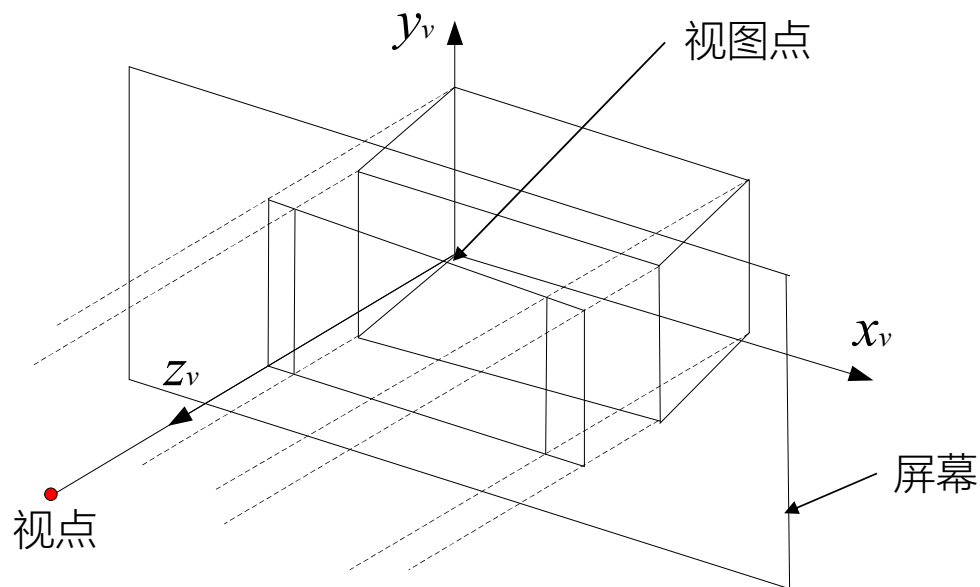


透视投影和平行投影

Perspective projection & Parallel projection



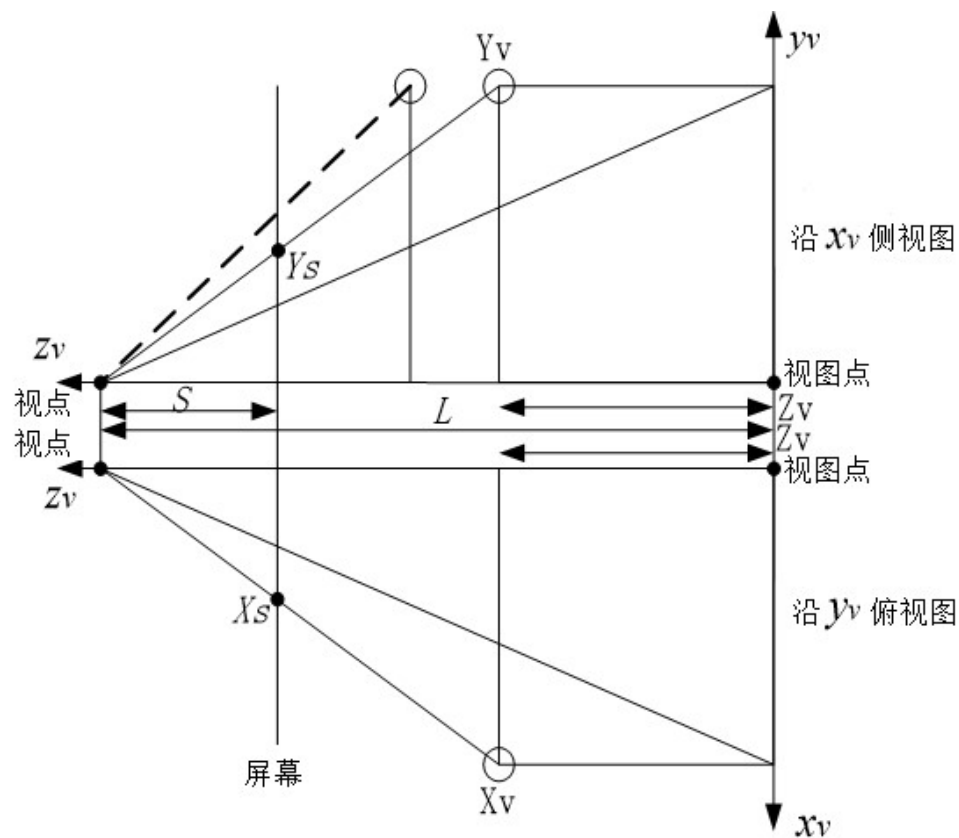
透视投影：模型上的所有点都连接到投影中心（投影中心通常位于从视图点到视点的直线上），这些线与屏幕之间的交点构成投影图像。



平行投影：平行线从物体上由视点定义的方向上的所有点投射，这些线与屏幕之间的交点构成图像。

观察坐标系

Viewing coordinate system



投影点的计算公式:

➤ 视图点相对于观察坐标系的坐标记为 X_v 、 Y_v 和 Z_v ;

➤ X_s 和 Y_s 是在屏幕的水平和垂直方向上测量的标记点与 Z_v 轴与屏幕相交的点的投影距离;

➤ 应用相似三角形关系, 给出了 X_s 和 Y_s 计算结果:

$$X_s = \frac{S}{L - Z_v} X_v$$

$$Y_s = \frac{S}{L - Z_v} Y_v$$

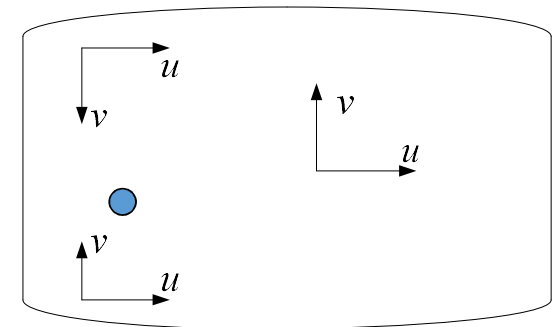
设备坐标系统和虚拟设备坐标系统

Device and virtual device coordinate systems

设备坐标系 - 用于在显示设备上定义位置。

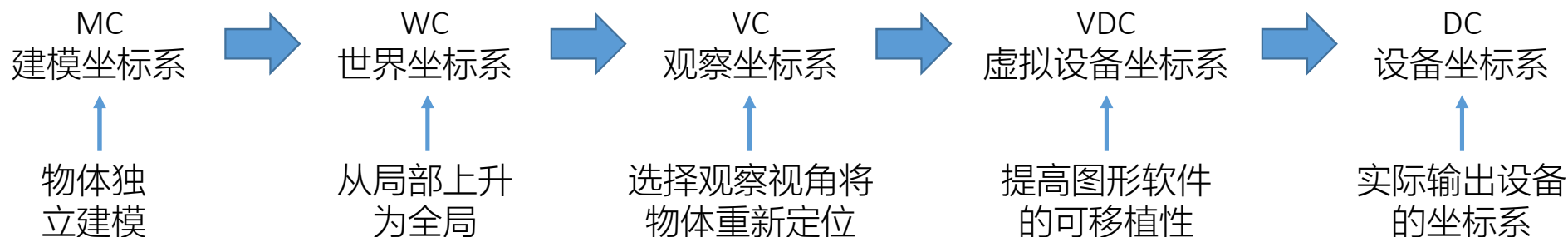
存在问题：不同的图形设备可能有不同的设备坐标系，在显示设备上的相同位置可能有不同的 u 和 v 二维坐标值。

虚拟设备坐标系 - 只存在于一个图形程序员的定义中。参照虚拟设备坐标系的值所指定的点，无论图形设备的类型如何，都将始终占据显示设备相同的位置。



设备坐标系

常用坐标系的转换关系



■ 用户利用各种输入设备进行交互绘图的基本操作步骤如下：

- ① 在局部坐标系中建立物体的几何模型；
- ② 将单个物体进行组装，形成世界坐标系中的全局模型；
- ③ 确定观察点位置，设定观察坐标系的位置；
- ④ 确定显示范围，选择对象的可见区域；
- ⑤ 确定图形显示设备上的视区。

目录

CONTENT

01

计算机图形学概述

Graphics libraries

02

坐标系统

Coordinate systems

03

图形变换

Transformation matrix

04

图形裁剪

hidden-surface removal

05

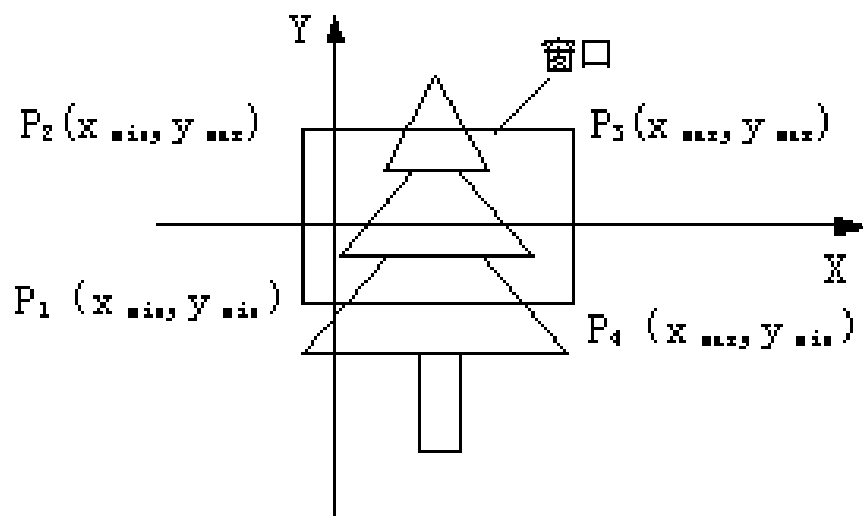
图形绘制

Rendering

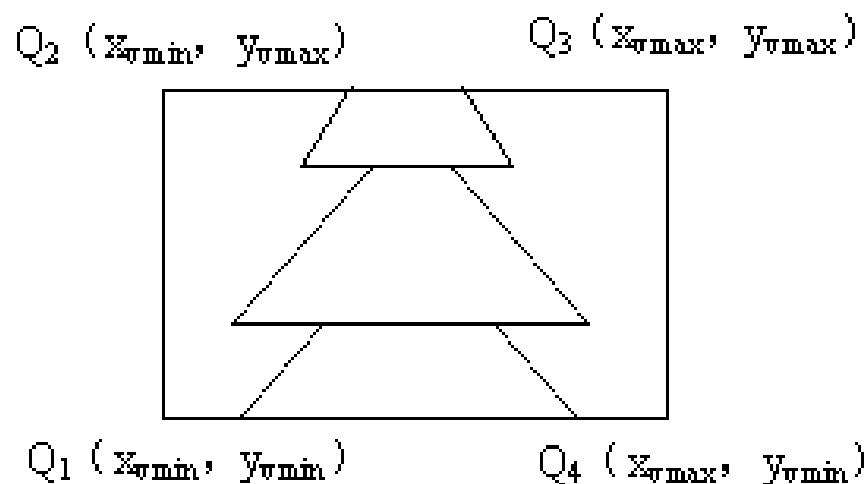
图形变换

■ 窗口与视口

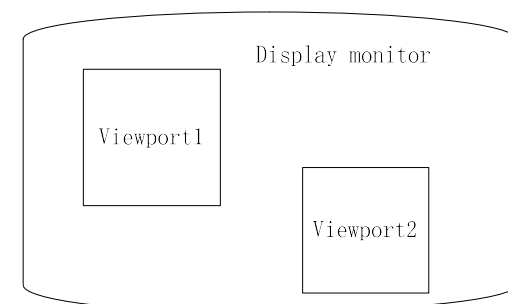
- 窗口（window）：窗口是用户在对象图形上选定的局部观察区域。
- 视口（view port）：在图形设备上，用来复制窗口内容的矩形区域称为视口。



(a) 定义窗口



(b) 视口图像



图形变换

■ 窗口、视口变换

- 将窗口内某一点W (x_w, y_w) 画在视口的指定位置V (x_v, y_v) 上须进行窗、视变换
- 窗口、视口变换的方程式:

$$\begin{cases} x_v = \frac{(x_w - x_{wmin})(x_{vmax} - x_{vmin})}{x_{wmax} - x_{wmin}} + x_{vmin} \\ y_v = \frac{(y_w - y_{wmin})(y_{vmax} - y_{vmin})}{y_{wmax} - y_{wmin}} + y_{vmin} \end{cases}$$

- 结论:

- ① 视口不变, 窗口缩小或放大时, 视口显示的图形会相应地放大或缩小;
- ② 窗口不变, 视口缩小或放大时, 视口中显示的图形将相应地缩小或放大;
- ③ 视口的纵横比不等于窗口的纵横比时, 视口中显示的图形将有伸缩变化;
- ④ 窗口与视口大小相同、坐标原点也相同时, 视口中显示的图形不变。

■ 图形变换的数学基础

2D/3D点的向量表示:

$$2D: p = \begin{bmatrix} x \\ y \end{bmatrix}$$

$$3D: p = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

用齐次坐标表示:

$$2D: p = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$3D: p = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

齐次坐标表示法: 由 $n+1$ 维向量表示一个 n 维向量

优点:

(1) 提供了用矩阵运算把二维, 三维甚至高维空间中的一个点集从一个坐标系变换到另一个坐标系的有效方法;

(2) 可以表示无穷远点: $n+1$ 维中, $h=0$ 的齐次坐标实际上表示了一个 n 维的无穷远点。对二维齐次坐标 $[a,b,h]$, 当 h 趋向0时, 表示了直线 $ax+by=0$, 即在直线上的连续点逐渐趋于无穷远。

应用: 通过齐次坐标, 我们可以将二维变换通过矩阵的形式统一起来:

$$T_{2D} = \begin{bmatrix} a & d & | & g \\ b & e & | & h \\ \hline c & f & | & i \end{bmatrix}$$

从变换功能上,
可以把 T_{2D} 分
为四个子矩阵:

$$\begin{bmatrix} a & d \\ b & e \end{bmatrix} \Rightarrow \text{缩放、旋转、对称、错切等}$$

$$\begin{bmatrix} c & f \end{bmatrix} \Rightarrow \text{投影} \quad \begin{bmatrix} g \\ h \end{bmatrix} \Rightarrow \text{平移}$$

$$[i] \Rightarrow \text{对整体图形作伸缩变换}$$

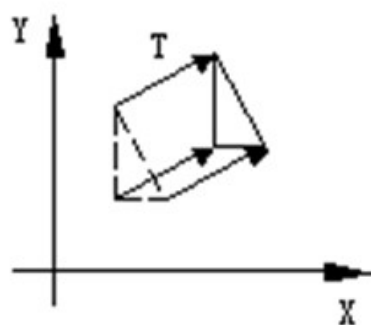
图形变换

■ 二维图形的几何变换

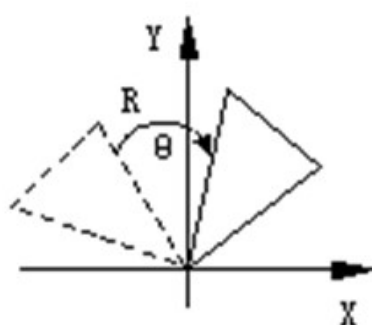
- 二维点、直线和平面的齐次坐标形式分别为：

二维点： $P = [x \ y \ 1]$ ； 二维直线： $L = \begin{bmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \end{bmatrix}$ ； 二维平面： $M = \begin{bmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{bmatrix}$

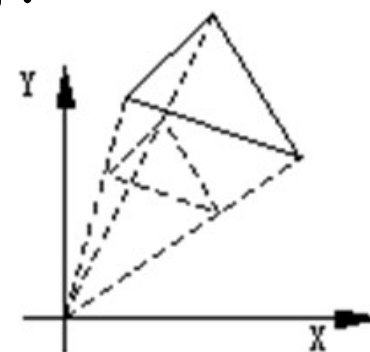
- 常见的二维图形几何变换包括平移、旋转和缩放等：



(a) 二维平移变换



(b) 二维旋转变换



(c) 二维缩放变换

图形变换

(1) 二维平移变换

平移变换的矩阵运算为：

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = \begin{bmatrix} x_1 + t_x \\ y_1 + t_y \\ 1 \end{bmatrix}$$

其中， $\mathbf{T}_m = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$ 为平移变换矩阵。

图形变换

(2) 二维旋转变换

旋转变换是指图形绕原点在xy平面内旋转 θ 角的变换过程。一般地，逆时针方向为正方向。以坐标原点为旋转基准点的旋转变换的矩阵运算为：

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = \begin{bmatrix} x_1 \cos \theta - y_1 \sin \theta \\ x_1 \sin \theta + y_1 \cos \theta \\ 1 \end{bmatrix}$$

图形变换

(3) 二维缩放变换

二维缩放变换按比例改变图形的尺寸。多边形的缩放变换可通过将每个顶点的坐标值 (x_1, y_1) 乘以缩放系数 S_x 和 S_y 以产生变换的坐标 (x_2, y_2) 来实现，其中 S_x 为在 x 方向的缩放系数， S_y 为 y 方向的缩放系数。缩放变换的矩阵运算为：

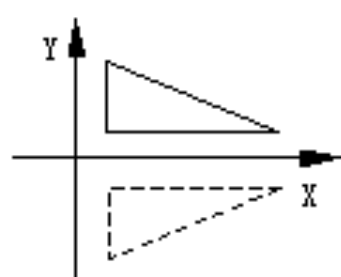
$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = \begin{bmatrix} x_1 S_x \\ y_1 S_y \\ 1 \end{bmatrix}$$

缩放系数 S_x 和 S_y 为正数。缩放系数小于1时，为缩小变换；缩放系数大于1时，为放大变换； $S_x = S_y$ 时，为一致性缩放； $S_x \neq S_y$ 时，为差值缩放。一般地，物体经缩放变换，既有大小的变化，又有位置改变。

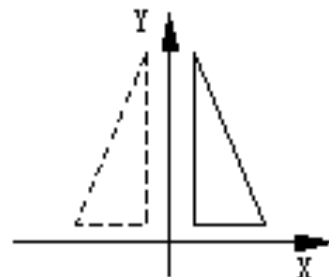
图形变换

(4) 二维对称变换

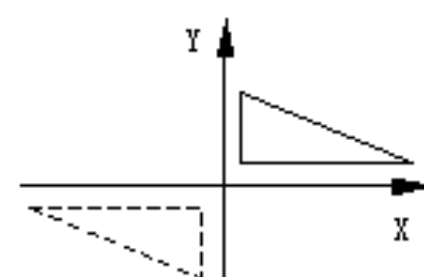
$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & 0 \\ d & e & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = \begin{bmatrix} ax_1 + by_1 \\ dx_1 + ey_1 \\ 1 \end{bmatrix}$$



(a) X轴对称

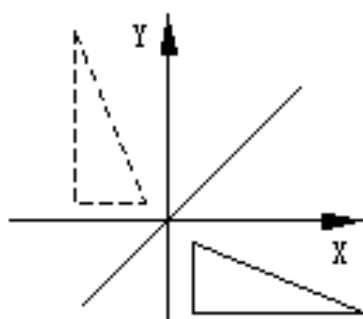


(b) Y轴对称

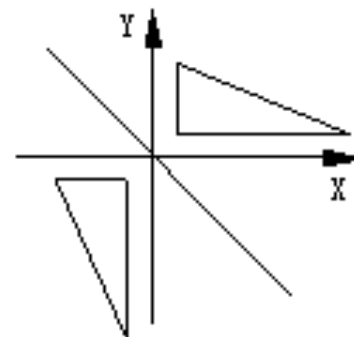


(c) 原点对称

对称变换的常见形式



(d) X=Y 直线对称



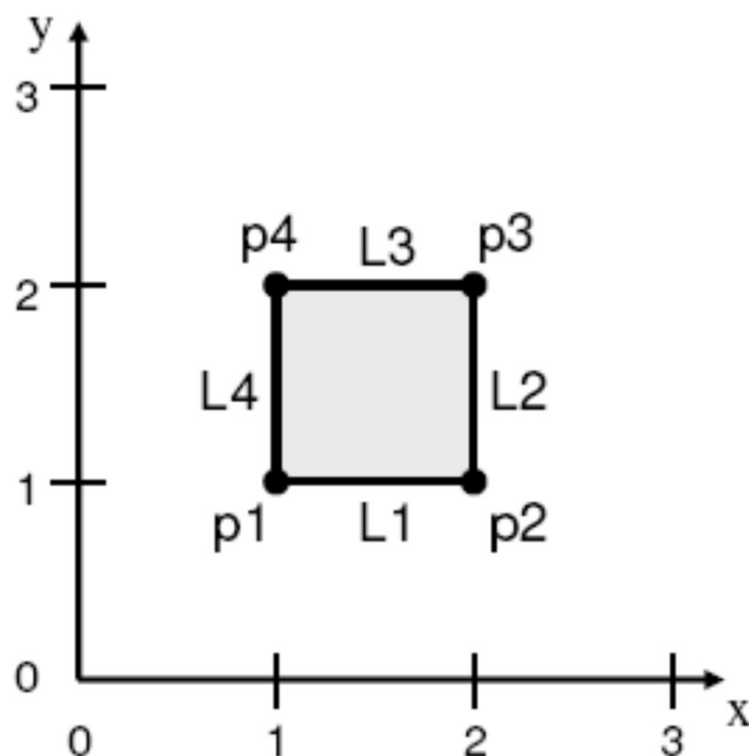
(e) X=-Y 直线对称

图形变换

(5) 复合变换

- 在基本几何变换中，旋转和缩放功能都是按相对于坐标原点进行的变换，而相对于其他参考点的旋转和缩放，则需要一组顺序的变换操作。
- 在实际应用中，一个图形可能要经过多次基本变换（如平移、旋转和缩放），才能满足建模要求，称之为复合变换。
- 复合变换就是指对图形作一次以上的基本几何变换的组合。利用齐次坐标矩阵，可以将任意顺序的变换矩阵依次相乘，形成变换矩阵的乘积，再与原始坐标相乘，从而获得变换的最终坐标值。

平面正方形表示：



$$L1 = \langle p1, p2 \rangle$$

$$L2 = \langle p2, p3 \rangle$$

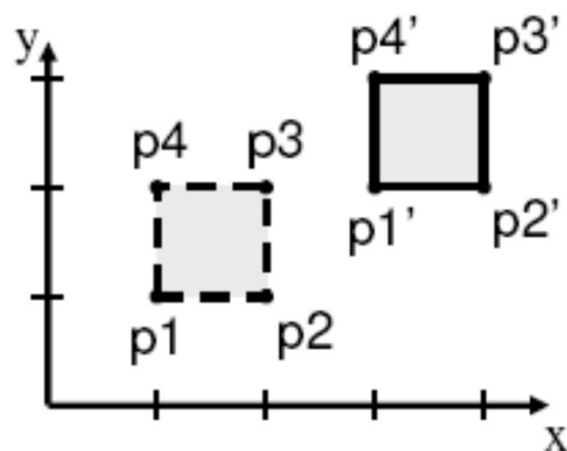
$$L3 = \langle p3, p4 \rangle$$

$$L4 = \langle p4, p1 \rangle$$

$$p1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \quad p2 = \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}$$

$$p3 = \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix} \quad p4 = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$$

正方形的平移变换



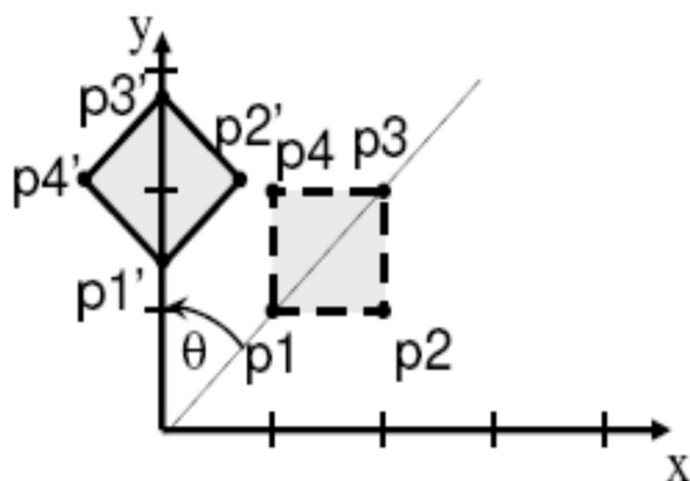
$$p1' = \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix} \quad p2' = \begin{bmatrix} 4 \\ 2 \\ 1 \end{bmatrix}$$

$$p3' = \begin{bmatrix} 4 \\ 3 \\ 1 \end{bmatrix} \quad p4' = \begin{bmatrix} 3 \\ 3 \\ 1 \end{bmatrix}$$

$$p' = Mp = Trans(\Delta x, \Delta y) \cdot p = \begin{bmatrix} 1 & 0 & \Delta x \\ 0 & 1 & \Delta y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$p1' = Trans(2, 1) \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix}$$

绕原点的旋转变换



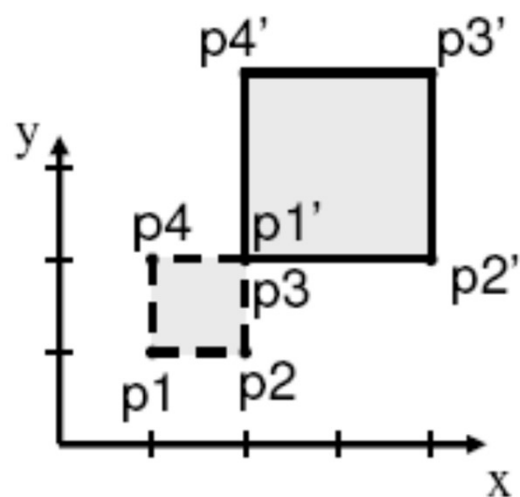
$$p1' = \begin{bmatrix} 0 \\ 1.414 \\ 1 \end{bmatrix} \quad p2' = \begin{bmatrix} .707 \\ 2.121 \\ 1 \end{bmatrix}$$

$$p3' = \begin{bmatrix} 0 \\ 2.828 \\ 1 \end{bmatrix} \quad p4' = \begin{bmatrix} -.707 \\ 2.121 \\ 1 \end{bmatrix}$$

$$p' = Mp = Rot(\theta) \cdot p = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$p1' = Rot(45^\circ) \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} .707 & -.707 & 0 \\ .707 & .707 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1.414 \\ 1 \end{bmatrix}$$

比例变换



$$p1' = \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix}$$

$$p3' = \begin{bmatrix} 4 \\ 4 \\ 1 \end{bmatrix}$$

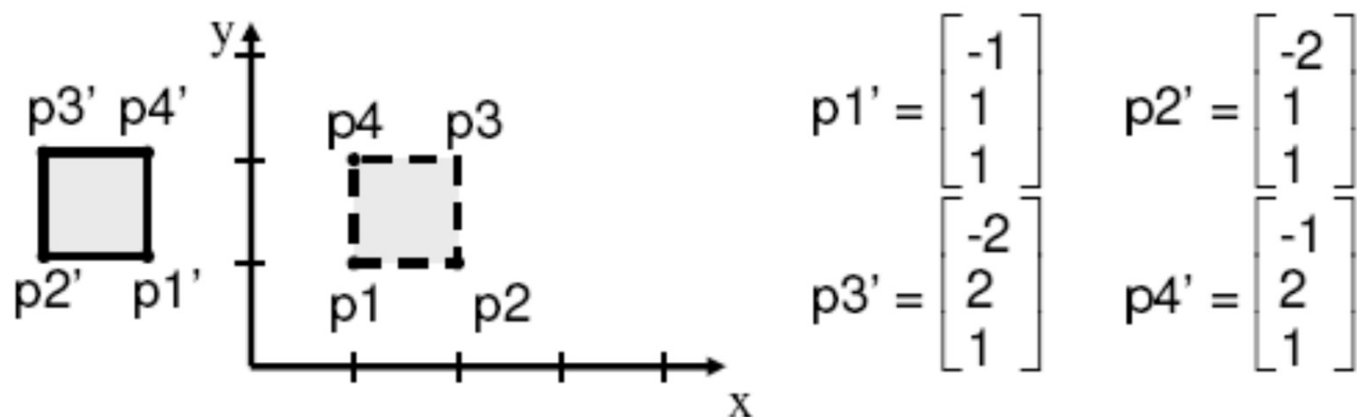
$$p2' = \begin{bmatrix} 4 \\ 2 \\ 1 \end{bmatrix}$$

$$p4' = \begin{bmatrix} 2 \\ 4 \\ 1 \end{bmatrix}$$

$$p' = Mp = Scale(sx, sy) \bullet p = \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$p1' = Scale(2, 2) \bullet \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix}$$

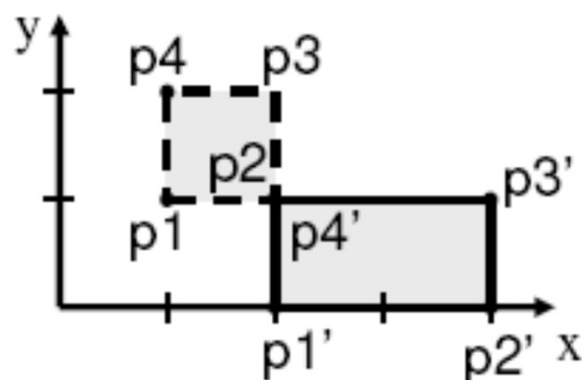
对称变换



$$p' = \mathbf{M}p = \text{MirrorX} \bullet p = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$p1' = \text{MirrorX} \bullet \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \\ 1 \end{bmatrix}$$

复合变换



$$p1' = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix} \quad p2' = \begin{bmatrix} 4 \\ 0 \\ 1 \end{bmatrix}$$

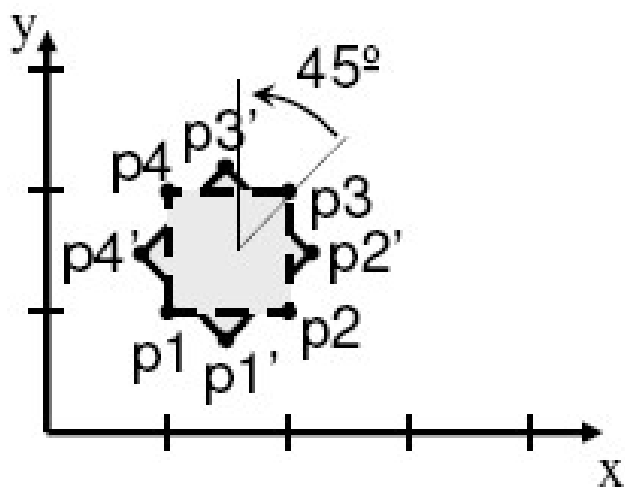
$$p3' = \begin{bmatrix} 4 \\ 1 \\ 1 \end{bmatrix} \quad p4' = \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}$$

$$p' = M_2 M_1 p = Trans(0, -1) \bullet Scale(2, 1) \bullet p$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$p1' = Trans(0, -1) \bullet Scale(2, 1) \bullet \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}$$

相对于任意点的旋转45度变换:



1.

2.

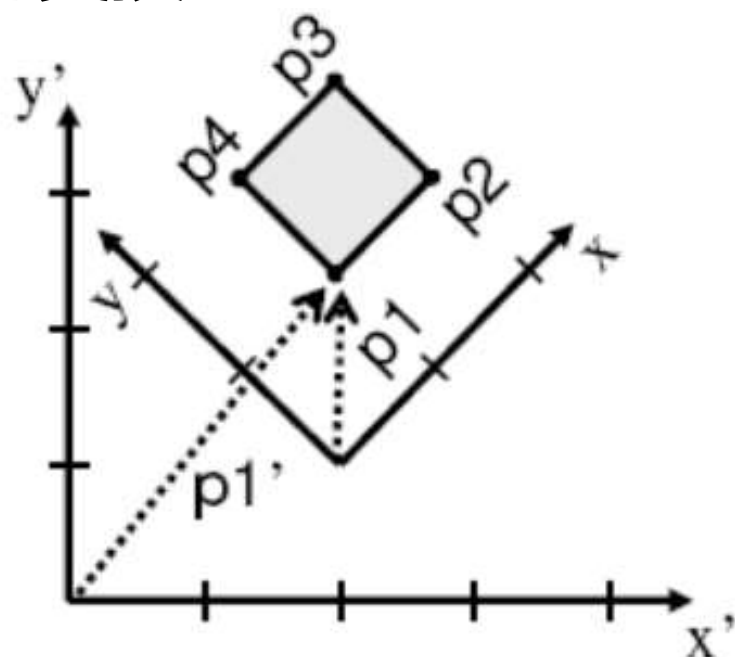
3.

$$p' = M_3 M_2 M_1 p$$

=

$$= \begin{bmatrix} & \\ & \\ & \\ & \end{bmatrix} \begin{bmatrix} & \\ & \\ & \\ & \end{bmatrix} \begin{bmatrix} & \\ & \\ & \\ & \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

映射变换：



$$p1' = \begin{bmatrix} ? \\ ? \\ 1 \end{bmatrix} \quad p2' = \begin{bmatrix} ? \\ ? \\ 1 \end{bmatrix}$$

$$p3' = \begin{bmatrix} ? \\ ? \\ 1 \end{bmatrix} \quad p4' = \begin{bmatrix} ? \\ ? \\ 1 \end{bmatrix}$$

$p1', p2', p3', p4'$ 在 $X' \sim Y'$ 坐标系中与原始的 $p1, p2, p3, p4$ 有相同的坐标值

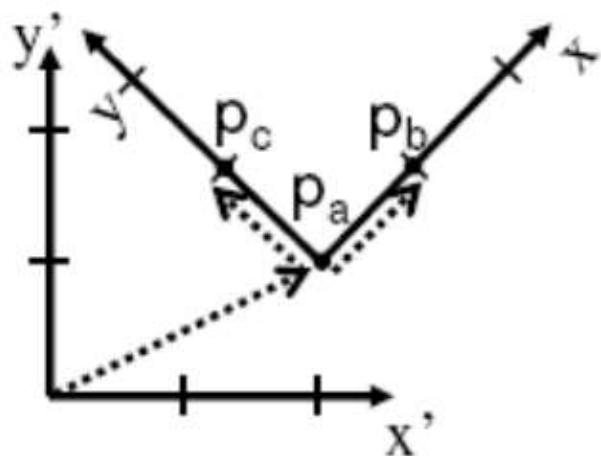
映射方程为：

$$p' = Tp = \begin{bmatrix} n_x & o_x & p_x \\ n_y & o_y & p_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

如何求解映射矩阵？

$$\text{代入 } p_a = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\text{代入 } p_b = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$



$$p_a' = \begin{bmatrix} n_x & o_x & p_x \\ n_y & o_y & p_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} p_x \\ p_y \\ 1 \end{bmatrix}$$

$$p_b' = \begin{bmatrix} n_x & o_x & p_x \\ n_y & o_y & p_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} n_x + p_x \\ n_y + p_y \\ 1 \end{bmatrix}$$

$$p_b' - p_a' = \begin{bmatrix} n_x + p_x \\ n_y + p_y \\ 1 \end{bmatrix} - \begin{bmatrix} p_x \\ p_y \\ 1 \end{bmatrix} = \begin{bmatrix} n_x \\ n_y \\ 0 \end{bmatrix}$$

$$\text{类似, } p_c' - p_a' = \begin{bmatrix} o_x \\ o_y \\ 0 \end{bmatrix}$$

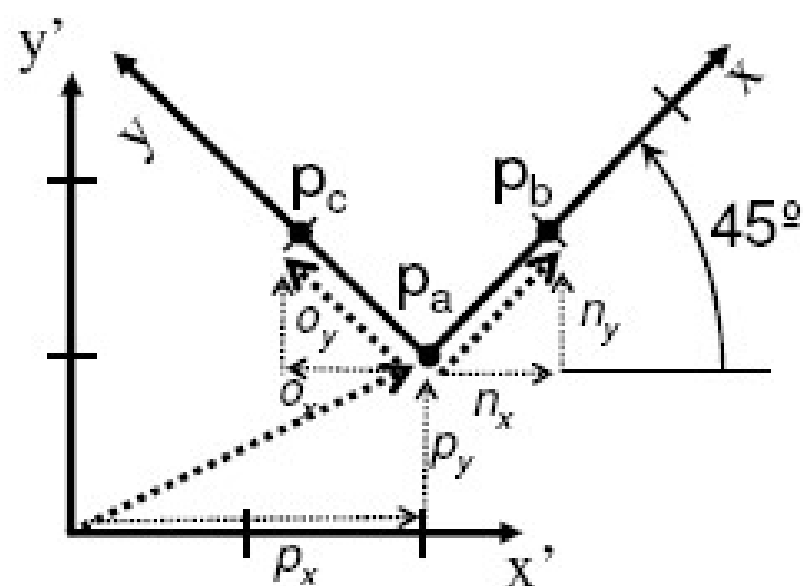
求解得到矩阵M

$$\begin{bmatrix} p_x \\ p_y \\ 1 \end{bmatrix} = \mathbf{p}_a' = \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}$$

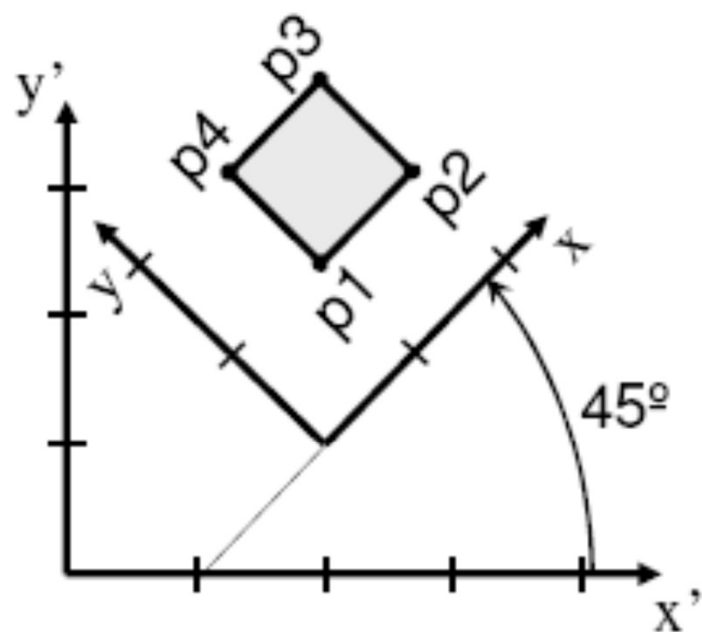
$$\begin{bmatrix} n_x \\ n_y \\ 0 \end{bmatrix} = \mathbf{p}_b' - \mathbf{p}_a' = \begin{bmatrix} .707 \\ .707 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} o_x \\ o_y \\ 0 \end{bmatrix} = \mathbf{p}_c' - \mathbf{p}_a' = \begin{bmatrix} -.707 \\ .707 \\ 0 \end{bmatrix}$$

$$\mathbf{M} = \begin{bmatrix} n_x & o_x & p_x \\ n_y & o_y & p_y \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} .707 & -.707 & 2 \\ .707 & .707 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$



映射变换验证：



$$p' = Tp = \begin{bmatrix} .707 & -.707 & 2 \\ .707 & .707 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$p1' = \begin{bmatrix} 2 \\ 2.414 \\ 1 \end{bmatrix} \quad p2' = \begin{bmatrix} 2.707 \\ 3.121 \\ 1 \end{bmatrix}$$

$$p3' = \begin{bmatrix} 2 \\ 3.828 \\ 1 \end{bmatrix} \quad p4' = \begin{bmatrix} 1.293 \\ 3.121 \\ 1 \end{bmatrix}$$

3D 变换矩阵

平移

$$p' = Mp = Trans(\Delta x, \Delta y, \Delta z) \bullet p = \begin{bmatrix} 1 & 0 & 0 & \Delta x \\ 0 & 1 & 0 & \Delta y \\ 0 & 0 & 1 & \Delta z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

缩放

$$p' = Mp = Scale(s_x, s_y, s_z) \bullet p = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3D 变换矩阵

对称

沿X方向镜像（关于YZ平面）

$$p' = Mp = MirrorX \bullet p =$$

$$\begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

沿Y方向镜像（关于ZX平面）

$$p' = Mp = MirrorY \bullet p =$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

沿Z方向镜像（关于XY平面）

$$p' = Mp = MirrorZ \bullet p =$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3D 变换矩阵

旋转

绕Z轴旋转

$$p' = Mp = RotZ(\theta) \cdot p = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 & 0 \\ \sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

绕X轴旋转

$$p' = Mp = RotX(\theta) \cdot p = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

绕Y轴旋转

$$p' = Mp = RotY(\theta) \cdot p = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

平移Translation

平移: 是指从一个位置到另一个位置的直线移动.

(1) 点

即从点 $P(x,y)$ 移动到 $P^*(x^*,y^*)$,在 X 轴上移动了 T_x 距离,在 Y 轴上移动了 T_y 距离

$$\begin{cases} X^* = X + T_x \\ Y^* = Y + T_y \end{cases}$$

(2) 多边形

多边形的平移变换是由构成多边形的每一条线的始、末点的坐标都 加上相应的平移量得到的;

(3) 圆、椭圆

它们的平移是需要通过移动其圆心坐标来实现的.

比例Scale

比例变换: 一个图形的尺寸,变换前,后成比例变化,即称之为比例变换. 通过比例因子 S_x 和 S_y 可以使多边形的坐标值放大或者缩小.

$$\begin{cases} X^* = X \cdot S_x \\ Y^* = Y \cdot S_y \end{cases}$$

当 $S_x, S_y < 1$ 时, 图形缩小;

$S_x, S_y = 1$ 时, 图形不变;

$S_x, S_y > 1$ 时, 图形放大;

$S_x \neq S_y$ 时, 图形沿两个坐标轴方向做非均匀的比例变换;

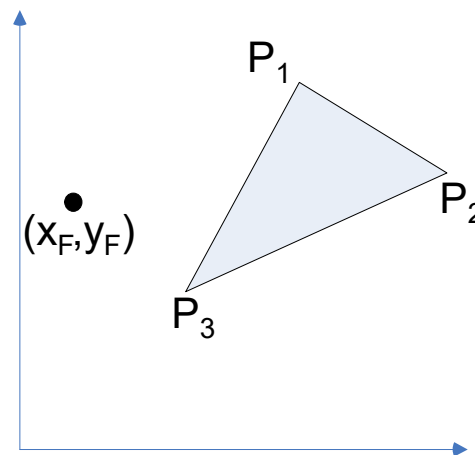
基点: 如果选择一个能控制图形比例变换的点,该点在变换后仍然保持不变,则称之为基点.基点的坐标用 (X_F, Y_F) 表示.

$$\begin{cases} X^* = X_F + (X - X_F) \cdot S_x \\ Y^* = Y_F + (Y - Y_F) \cdot S_y \end{cases}$$
$$\Rightarrow \begin{cases} X^* = X \cdot S_x + (1 - S_x) \cdot X_F \\ Y^* = Y \cdot S_y + (1 - S_y) \cdot Y_F \end{cases}$$

对图形中所有点来说

$$\begin{cases} (1 - S_x) \cdot X_F \\ (1 - S_y) \cdot Y_F \end{cases}$$

是一个常量

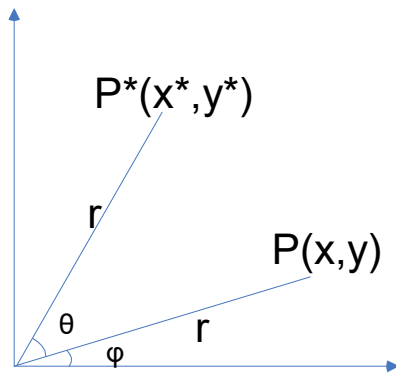


旋转Rotation

1. 绕原点旋转

如果图形上的点沿着图形轨迹变换,则称之为旋转变换,多边形的旋转变换需要定义每个顶点的旋转量.

点P旋转至P*,坐标原点为旋转中心



$$\begin{cases} X^* = r \cos(\phi + \theta) = r \cos \phi \cos \theta - r \sin \phi \sin \theta \\ Y^* = r \sin(\phi + \theta) = r \sin \phi \cos \theta + r \cos \phi \sin \theta \end{cases}$$

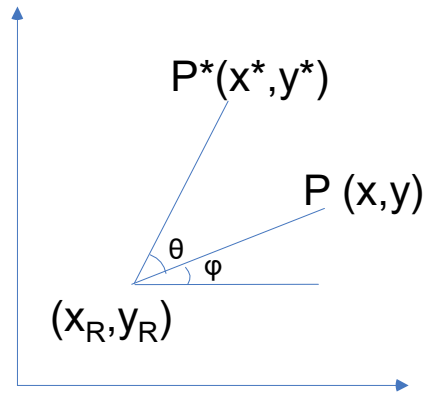
$$\Rightarrow \begin{cases} X^* = X \cos \theta - Y \sin \theta \\ Y^* = Y \cos \theta + X \sin \theta \end{cases}$$

$$\begin{cases} X = r \cos \phi \\ Y = r \sin \phi \end{cases}$$

旋转Rotation

2. 绕任意轴旋转

点P旋转至P*, 旋转中心为 (x_R, y_R)



$$\begin{cases} X^* = X_R + (X - X_R) \cos \theta - (Y - Y_R) \sin \theta \\ Y^* = Y_R + (Y - Y_R) \cos \theta + (X - X_R) \sin \theta \end{cases}$$

目录

CONTENT

01

计算机图形学概述
Graphics libraries

02

坐标系统
Coordinate systems

03

图形变换
Transformation matrix

04

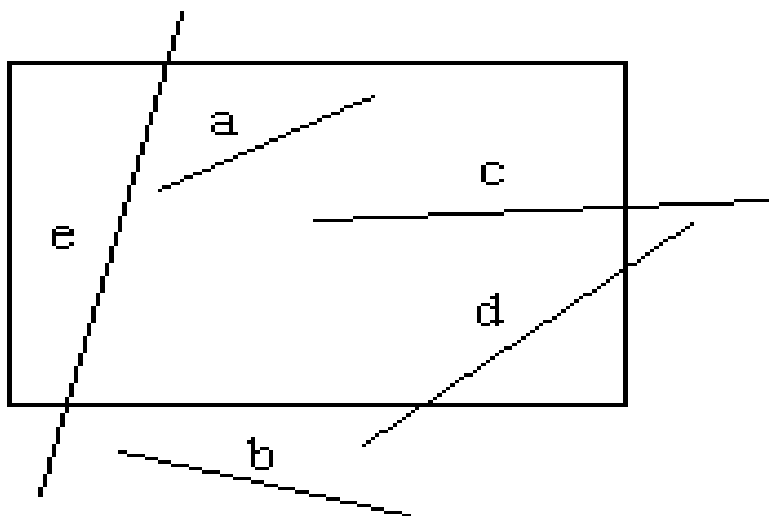
图形裁剪
hidden-surface removal

05

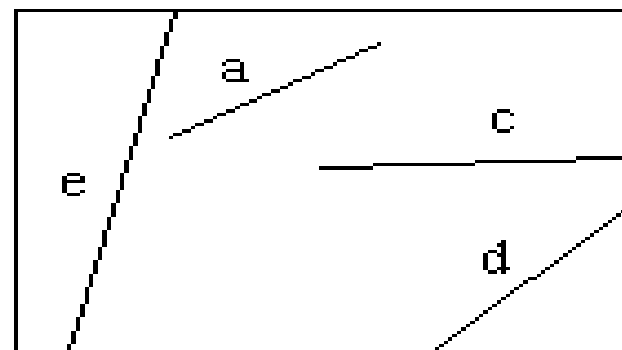
图形绘制
Rendering

图形裁剪

■ 线段的裁剪



(a) 裁剪前



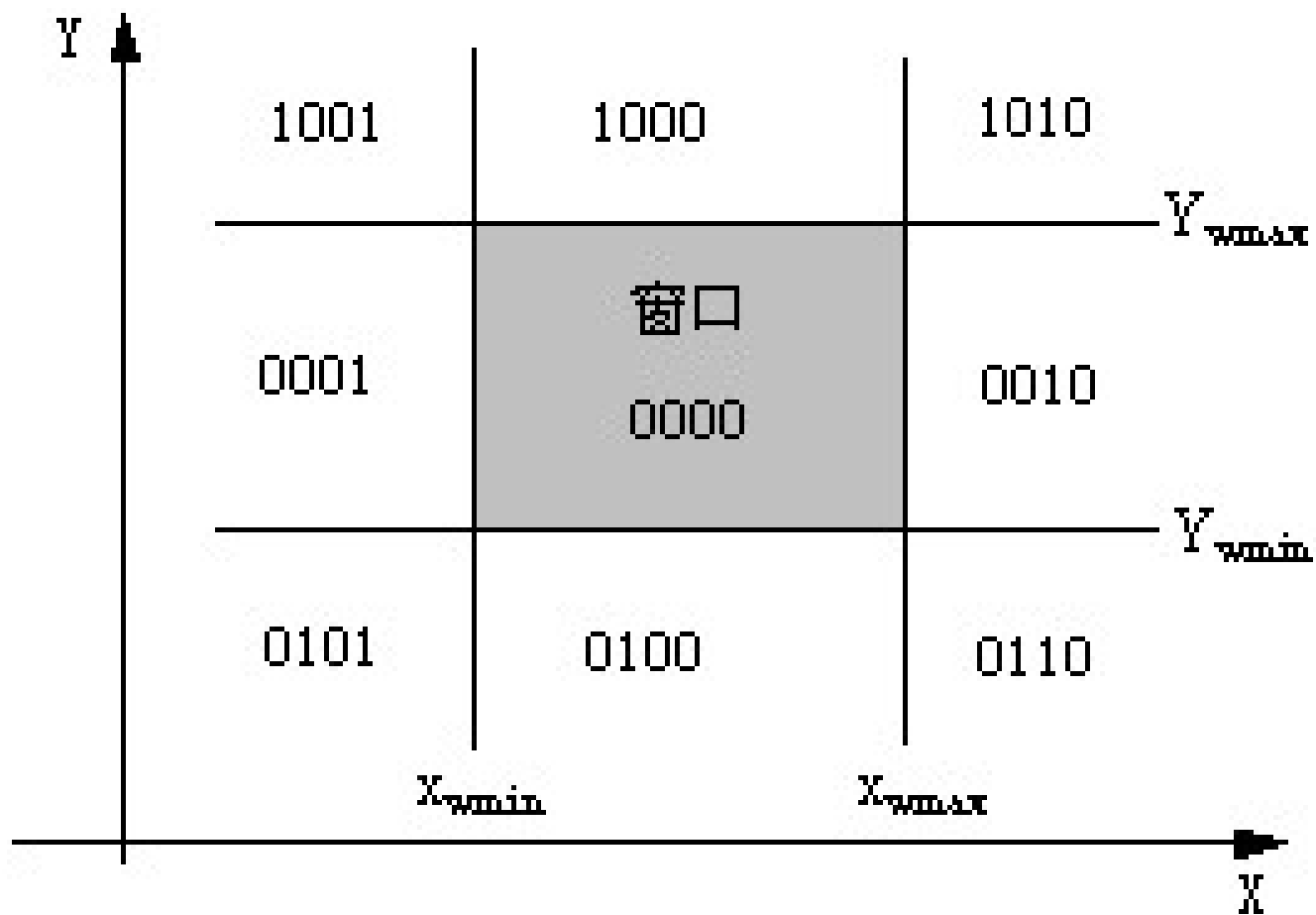
(b) 裁剪后

- 假定 x_{wmin} 、 x_{wmax} 、 y_{wmin} 、 y_{wmax} 为矩形裁剪窗口的4条边线位置，则判定点 (x, y) 是否位于裁剪窗口内的依据为：

$$\begin{cases} x_{wmin} \leq x \leq x_{wmax} \\ y_{wmin} \leq y \leq y_{wmax} \end{cases}$$

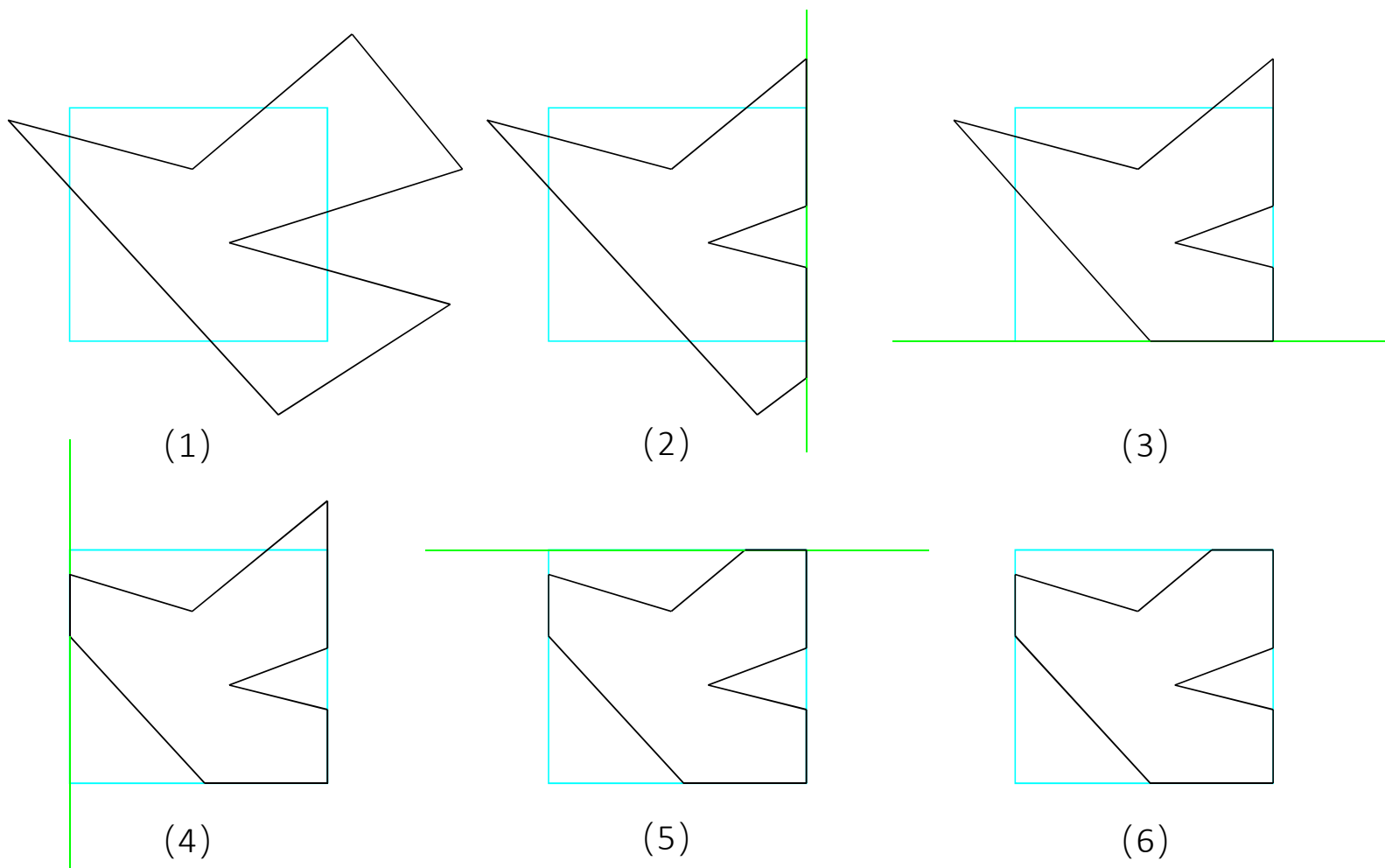
图形裁剪

■ Cohen-Sutherland 线段裁剪算法:



图形裁剪：多边形裁剪

■ 逐边裁剪法：



目录

CONTENT

01

计算机图形学概述

Graphics libraries

02

坐标系统

Coordinate systems

03

图形变换

Transformation matrix

04

图形裁剪

hidden-surface removal

05

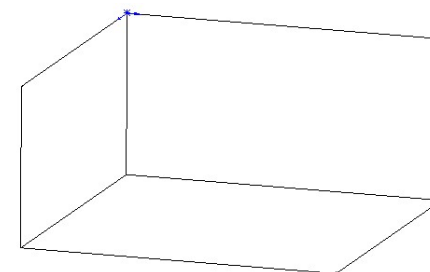
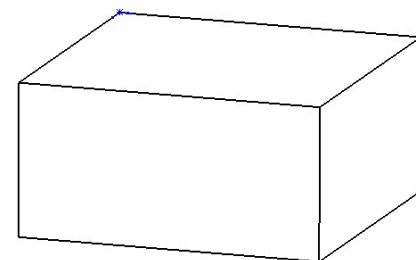
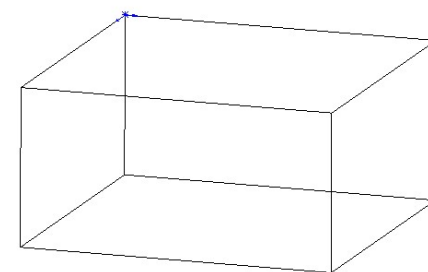
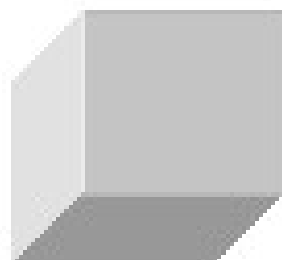
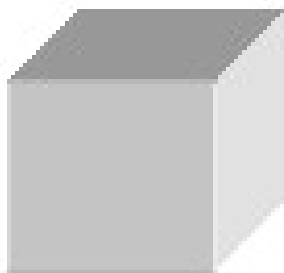
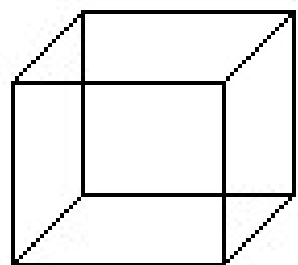
图形绘制

Rendering

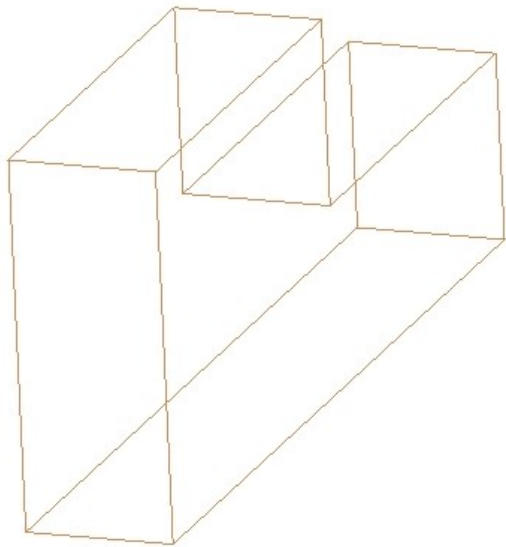
图形显示的渲染技术

■ 消除隐藏线及隐藏面

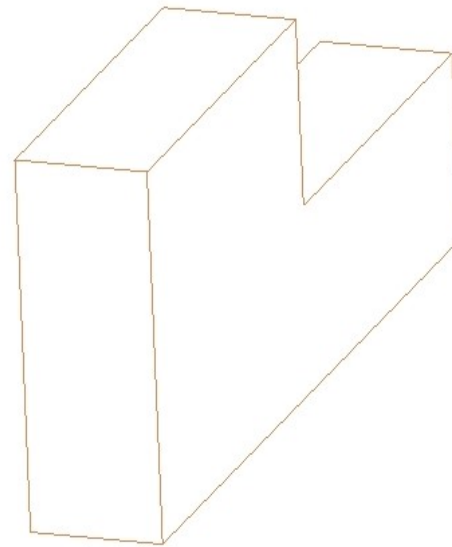
线框图形的二义性



图形消隐

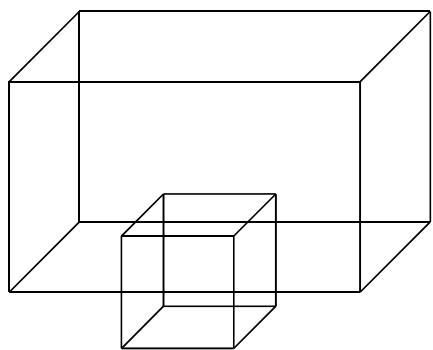


消隐前

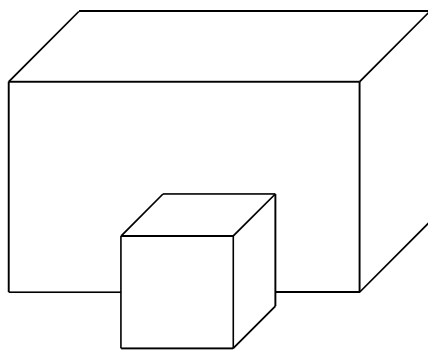


消隐后

图形消隐



(a) 消隐前



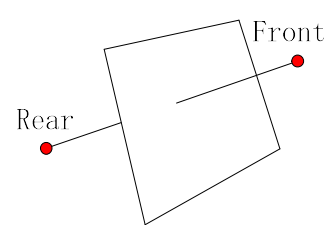
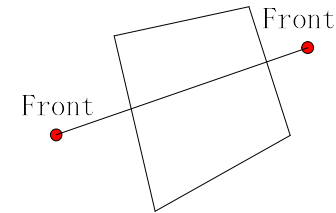
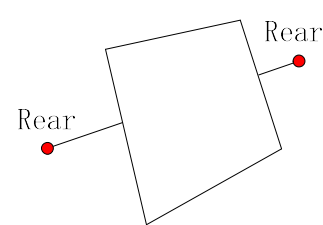
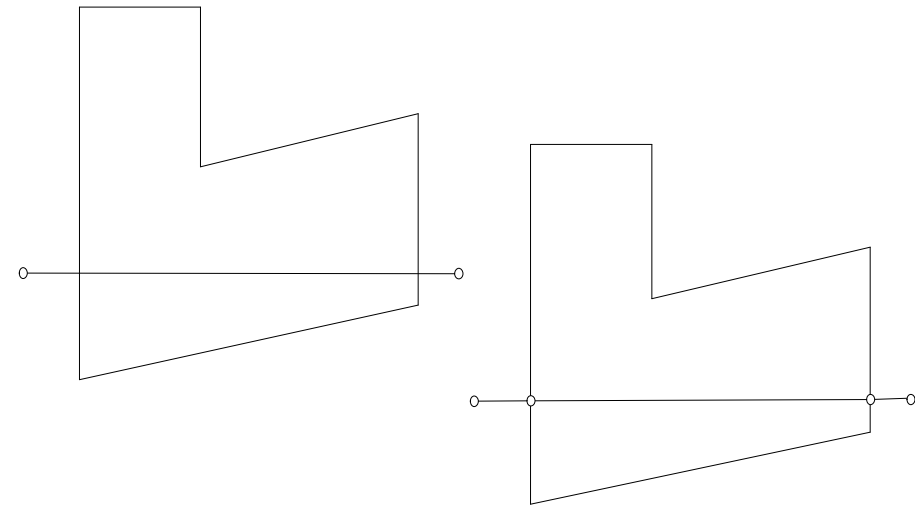
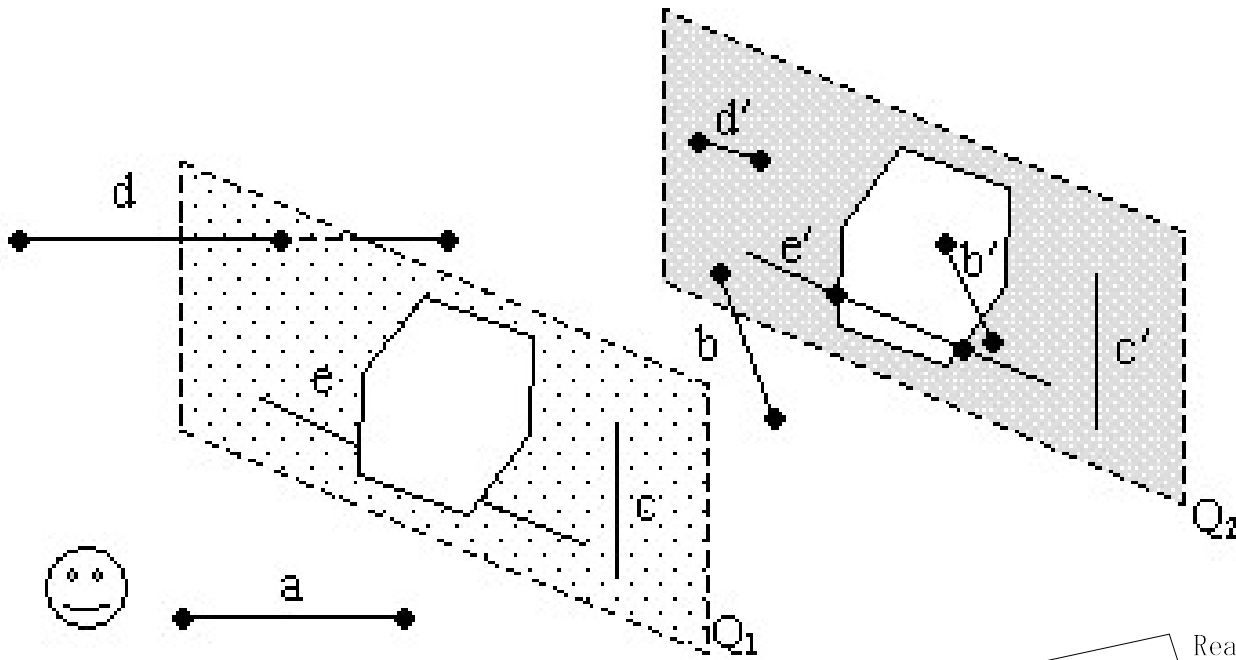
(b)消隐后

主要算法：

1. Back-face removal algorithm (背面消除法)
2. Z-buffer method (Z缓冲区算法)
3. Depth-sorting algorithm (深度排序法)
4. Painter's algorithm (画家算法)
5. Scan beam method (扫描线法)

图形显示的渲染技术

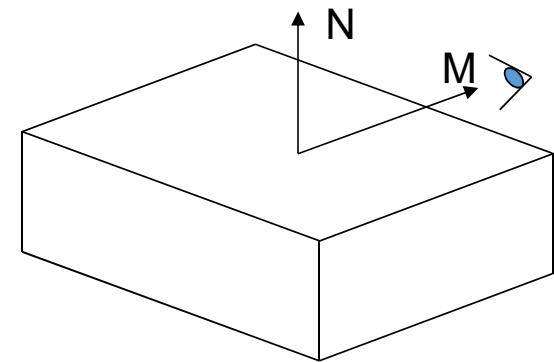
1. 消除隐藏线——直线段的遮挡判断



Three possible situations involving positions of faces and edges

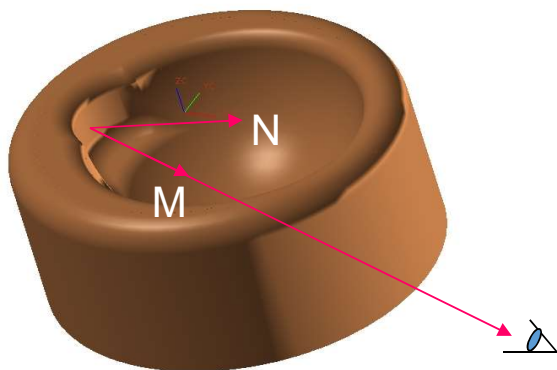
背面消除算法

- 基本思想:如果一个物体的外部方向上的法向量指向观察者, 那么它的面就会是可见的, 否则就会是不可见的。
- 如果 $M \bullet N > 0$, 那么该面可见
- 如果 $M \bullet N = 0$, 该面显示为一条线
- 如果 $M \bullet N < 0$, 该面不可见

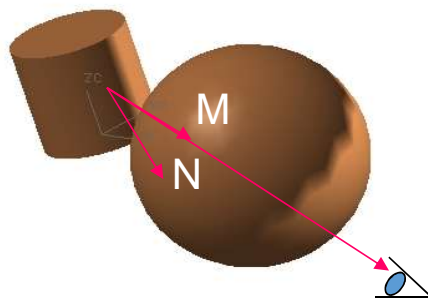


如果它的向外的法向量 N 在 M 的方向上有一个正分量, 那么顶面是可见的, M 是一个从面上的一个点到观察者的向量。

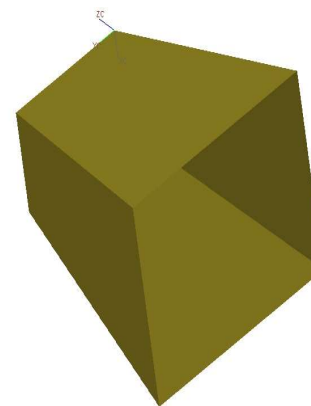
背面消除算法的缺点：只能应用于一个凸对象



a. 不能用于凹面对象，因为指向查看者的面可能会被同一对象的另一个面遮挡。



b. 不能用于一个以上的凸对象，当一个物体的面可以被另一个物体遮挡，即使面指向观察者

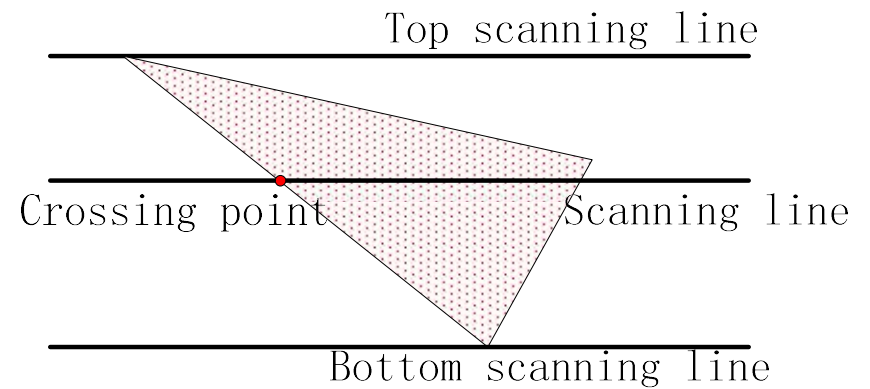
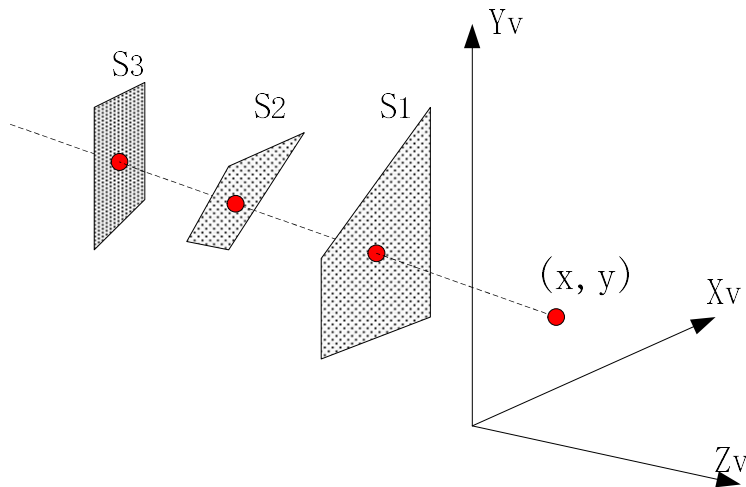


c. 不能用于这样的对象，其每个面的向外法向量不能被明确地确定。

z缓冲区方法

1. $\text{depth}(x,y)=0$, $\text{refresh}(xy)=I_{\text{background}}$

2. If $z > \text{depth}(x,y)$, then $\text{depth}(x,y)=z$, $\text{refresh}(x,y)=I_{\text{surf}}(x,y)$

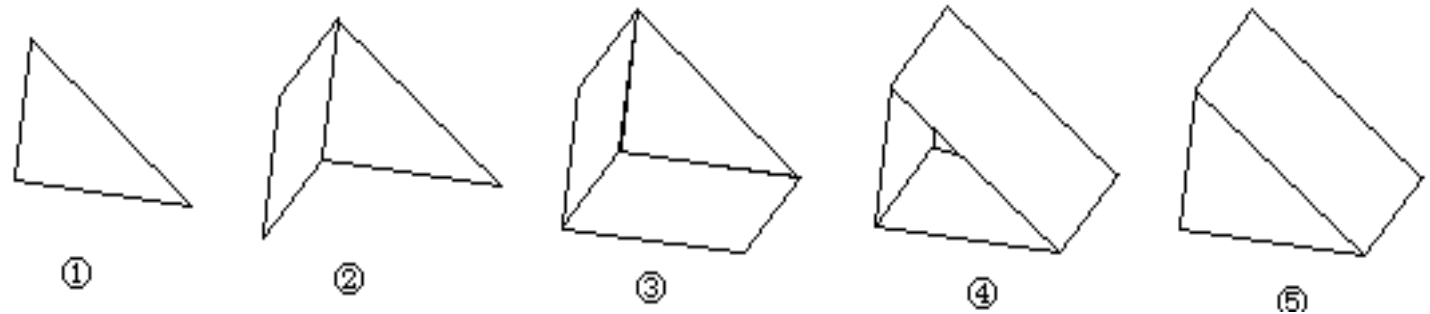


图形显示的渲染技术

2. 消除隐藏面——画家算法 (Depth-sorting, or painter's algorithm)

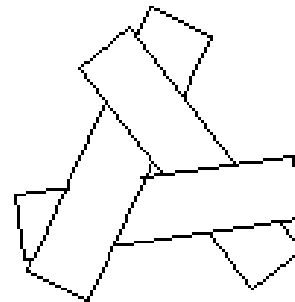
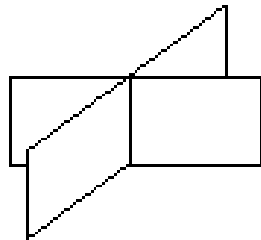
- **基本思路：**对象的表面按距离查看者的距离进行排序，并从最远的面到最近的面填充它们各自的颜色。如果较远的曲面必须占据屏幕中的相同区域，则它们会被较近的曲面自动隐藏。
- 曲面与观察者之间的距离是由面上的点相对于观察坐标系统的 z 坐标来测量的。也就是说， Z_v 坐标越大的点可以说更接近观察者。

画家算法的实现步骤



存在的问题

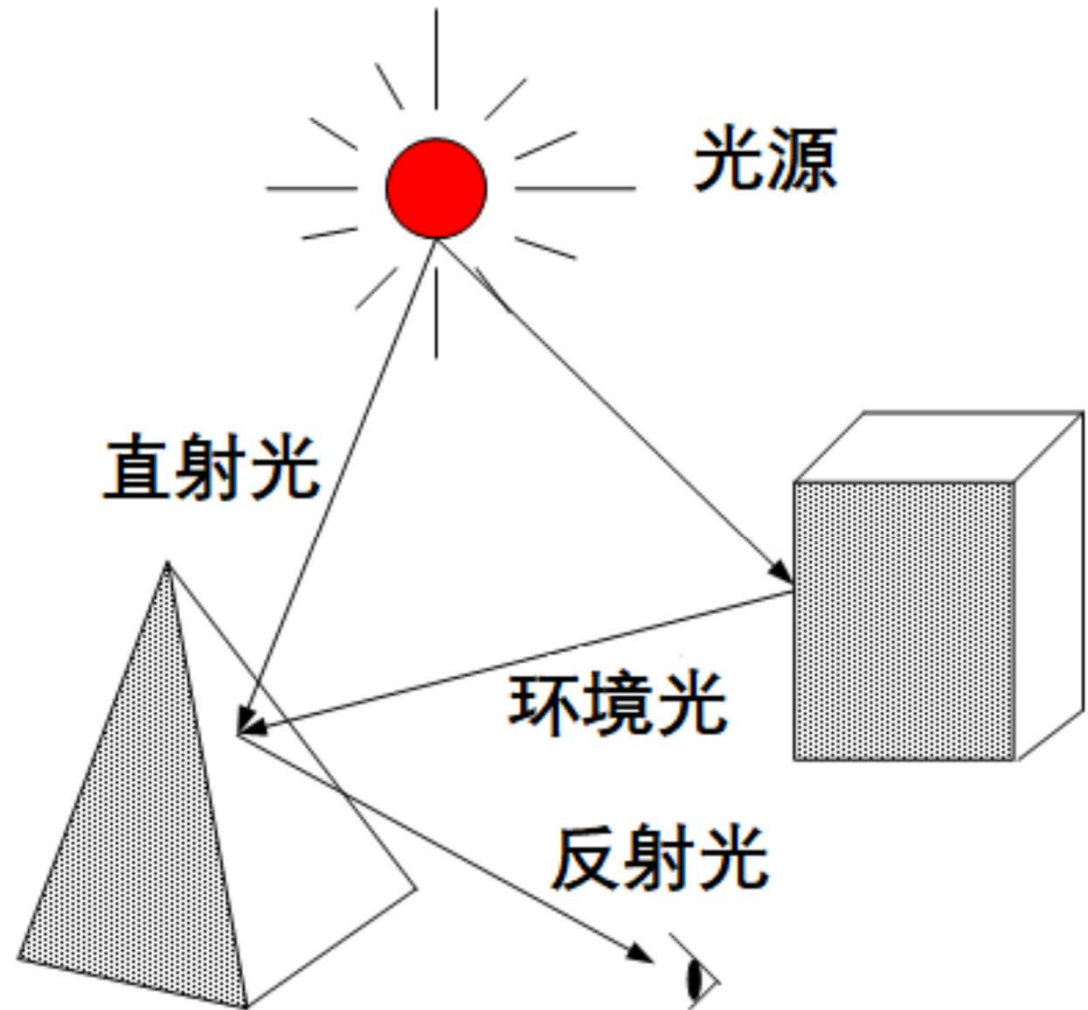
多边形交叉和重叠的情况:



图形显示的渲染技术

■ 颜色处理

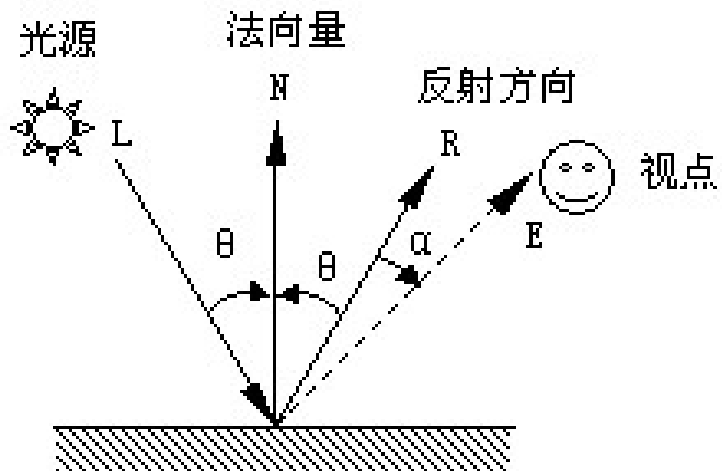
着色 (Shading)



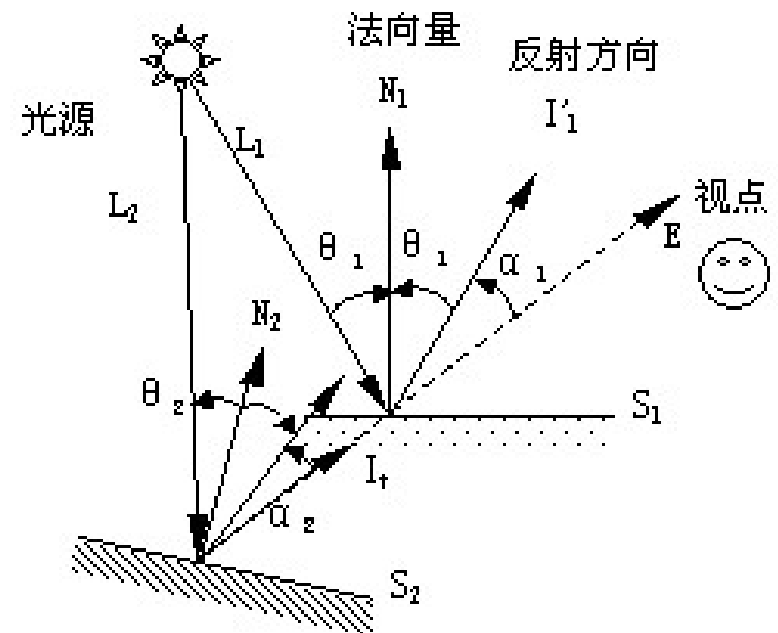
图形显示的渲染技术

■ 光照模型

基本光照模型



两个多面体的光照模型：光线的反射



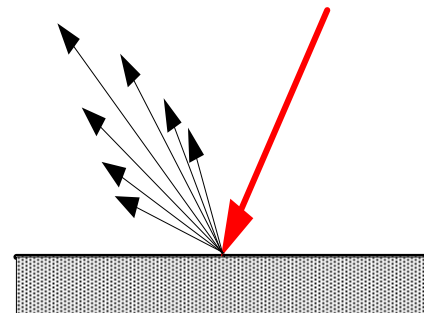
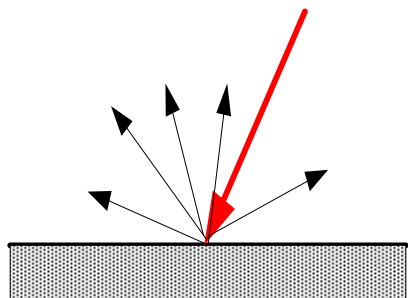
环境光Ambient light

强度 R_a ,

可以表示为 $R_a = K_d I_a$

其中, I_a —环境光的强度; K_d -表面的反射率系数。

漫反射和镜面反射

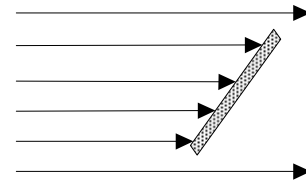
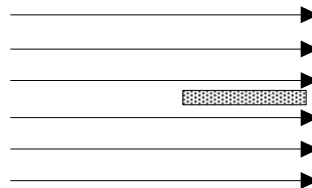
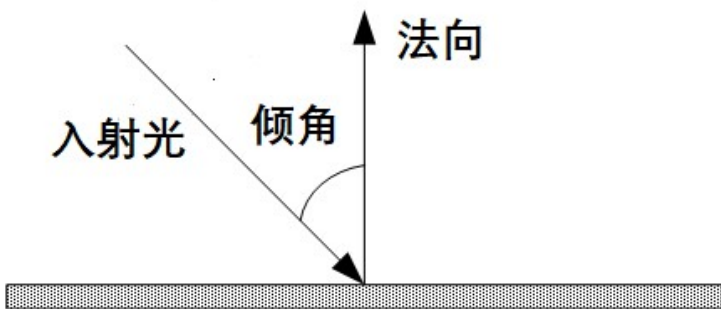


计算漫反射和镜面反射的强度

- 由点光源照射的某一点的反射可以近似为这两种反射强度之和。

漫反射的强度与入射角的余弦值成正比

用Lamber余弦定律计算



$$R_d = K_d E_p \cos \theta$$

$$\cos \theta = N \bullet L$$

$$R_d = K_d E_p (N \bullet L)$$

修正公式:

$$R_d = \frac{K_d E_p}{D + D_0} (N \bullet L)$$

R_d —漫反射强度

K_d — 反射率

E_p — 光源强度

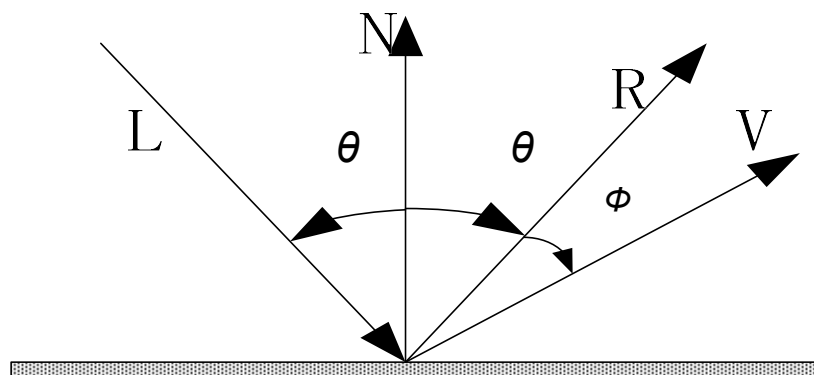
N — 关注点的单位法向量

L — 点到光源的单位向量

D — 点与观察点或观察者之间的距离

D_0 — 避免为0而添加

镜面反射强度的计算——phong模型



$$R_d = \frac{K_d E_p}{D + D_0} K_s (V \bullet R)^n \quad V \bullet R = \cos \phi$$

R — 反射光所在的小圆锥体轴（单位向量）

V — 视线向量（单位向量）

K_s — 镜面反射率

综合环境照明、漫反射和镜面反射

- 反射强度计算公式:

$$I = K_d I_d + \frac{E_p}{D + D_0} [K_d (N \bullet L) + K_s (V \bullet R)^n]$$

RGB分量:

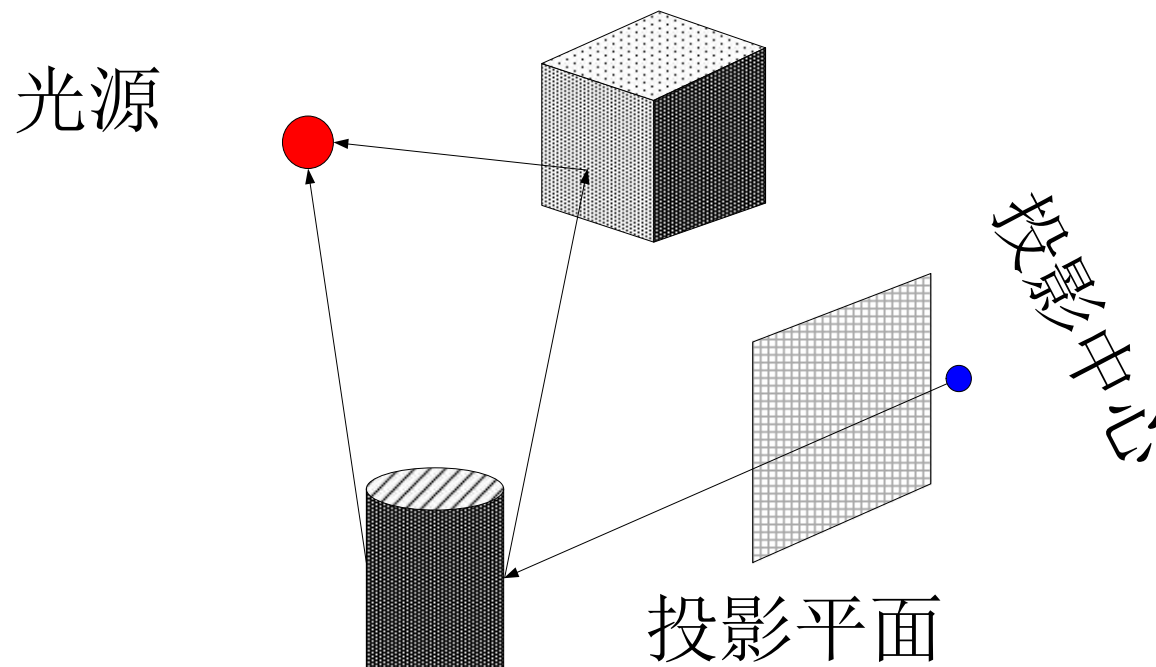
$$K_{dr} + K_{dg} + K_{db} = 1 \quad I_r = K_{dr} I_{dr} + \frac{E_{pr}}{D + D_0} [K_{dr} (N \bullet L) + K_s (V \bullet R)^n]$$

$$I_g = K_{dg} I_{dg} + \frac{E_{pg}}{D + D_0} [K_{dg} (N \bullet L) + K_s (V \bullet R)^n]$$

$$I_b = K_{db} I_{db} + \frac{E_{pb}}{D + D_0} [K_{db} (N \bullet L) + K_s (V \bullet R)^n]$$

Ks没有被分解，因为镜面反射对颜色几乎没有影响

光线跟踪Ray tracing



光线从光源向各个方向发射，每条光线都被追踪，直到它被投射到屏幕上的一个点上。

图形显示的渲染技术

■ 纹理处理

- 纹理（texture）是指物体表面的细节描述。例如：木材表面有木纹、织物的表面有纤维织造的纹理。

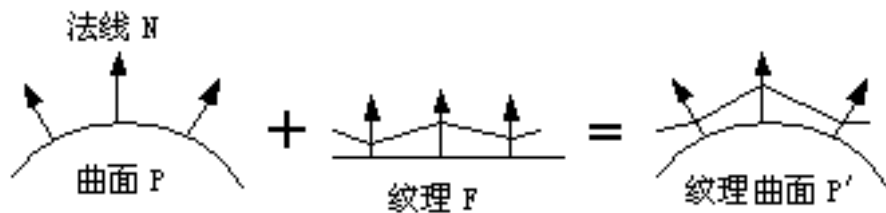
模型的纹理处理



利用斑马条纹检查模型的表面特性



纹理生成的基本原理





The End

THANKS

CAD & CAM

2⁰₂6